



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

LLM Agent-Driven Tool Selection and Query Reformulation

Yash Vardhan Thirani



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Data Engineering and Analytics

LLM Agent-Driven Tool Selection and Query Reformulation

Werkzeugauswahl und Anfrageumformulierung durch LLM-gesteuerte Agenten

Author:	Yash Vardhan Thirani
Supervisor:	Prof. Dr. Matthias Grabmair
Advisor:	Shanshan Xu
Submission Date:	December 09, 2024

I confirm that this Master's Thesis in Data Engineering and Analytics is my own work and I have documented all sources and material used.

Munich, December 09, 2024

Yash Vardhan Thirani

Acknowledgments

I would like to express my heartfelt gratitude to all those who supported me throughout the course of this thesis. First, I would like to thank Prof. Dr. Matthias Grabmair for the opportunity to complete my thesis at the Professorship for Legal Tech, and my advisor Shanshan Xu and Santosh Tokala for their invaluable guidance at all stages, right from the early stage of problem formulation, to research, to feedback and revisions, despite their own extensive commitments in different timezones. I would also like to extend my appreciation to Udo Knop of Bluesolve GmbH and my co-workers for help with the infrastructure and resources to perform experiments and support in letting me balancing the requirements of my working student job with the bandwidth required to write a thesis. Finally, I am deeply grateful to my parents in India and my friends for their patience and understanding through all the ups and downs of this challenging yet rewarding period.

Abstract

Although Large Language Models (LLMs) have demonstrated impressive capabilities for information seeking scenarios through their vast inherent knowledge and strong reasoning ability, they struggle to deal with queries that require real-time knowledge of events or specialist knowledge, often resulting in them generating misleading or false information, often termed ‘hallucinations’. To compensate for this limitation, recent research has explored integration of external tools and APIs, to augment the capabilities of LLMs in dealing with these real-time information seeking scenarios, in a typical Retrieval-augmented Generation (RAG) framework. While enhancing their problem solving capabilities, they also provide interpretability and foster user trust. Despite these advancements, they still struggle on multi-step queries. In this thesis, we investigate the utility of incorporating components such as Query Decomposition (breaking a complex query into a sequence of steps), Query Reformulation (rewriting the query to make it more concise and targeted during reflection) and Self-refine (providing feedback to iterate over an incorrect answer), through an autonomous multi-agent system framework, to assist the model in more precise retrieval and accurate generations in RAG framework.

We validate our proposed method against an existing framework called AnyTool which uses function calling and a hierarchical agent-based retrieval system to solve queries requiring tool use. We observe clear positive results that demonstrate an effectiveness of our proposed additions, especially in solving multi-part queries that require 2 or more external tools.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
1.1. Motivation	2
1.2. Research Questions	3
2. Fundamentals	5
2.1. Question Answering	5
2.2. Tool Use	6
2.3. Autonomous Agents	7
2.4. Reflection	8
2.5. Query Rewriting	9
2.5.1. Decomposition	10
2.5.2. Reformulation	11
3. Related Work	12
3.1. Reflection	12
3.1.1. LLaTrieval	12
3.1.2. Self-Refine	14
3.2. Query Rewriting	15
3.3. Tool Use	16
3.3.1. ToolLLM	17
3.4. LLM-powered Autonomous Agents	19
3.4.1. Function Calling	20
3.4.2. AnyTool	21
4. Methodology	25
4.1. Query Decomposition	25
4.1.1. System Prompt for Query Decomposer	27
4.2. Self-Refine	27

4.3. Query Reformulation Agent	29
4.3.1. System Prompt for Query Reformulator Agent	31
5. Experimental Setup	32
5.1. Tool repository	32
5.2. Datasets	34
5.2.1. Organization	34
5.2.2. Generation	34
5.2.3. Sampled subset of queries	35
5.3. Metrics	37
5.4. Model	38
6. Results and Errors	40
6.1. Overview across pooled queries	40
6.2. Queries grouped by number of required tools	40
6.2.1. Solvability	41
6.2.2. Pass Rate	42
6.3. Ablation Study	44
6.4. Statistical Significance	48
6.4.1. Overall Performance	49
6.4.2. Split by number of tools	50
6.5. Error Analysis	51
6.5.1. Error types	51
6.5.2. Retriever Errors	51
6.5.3. Solver Errors	52
7. Conclusion and Future Research	55
7.1. Conclusions	55
7.1.1. Influence of Query Decomposition:	55
7.1.2. Influence of Self-refine:	55
7.1.3. Influence of Query Reformulation with a fresh conversation history:	56
7.2. Future Research	56
7.2.1. Token consumption	56
7.2.2. Answer generation time	56
List of Figures	58
List of Tables	59

Contents

A. Additional Prompts	60
A.1. System Prompt for Rewrite Validity	60
A.2. System prompt for combining provisional answers into single final answer	61
Bibliography	62

1. Introduction

In recent years, Large Language Models (LLMs) have driven substantial progress in Natural Language Processing (NLP), showcasing a remarkable ability to understand and generate text with human-like fluency. Models like OpenAI's GPT-4 [1], Google's PaLM [2], and Meta's LLaMA [3] demonstrate how these advanced systems can decompose complex problems into structured steps, offer nuanced explanations, and provide insightful recommendations. By moving beyond simple text generation, these models leverage sophisticated reasoning to interpret queries, distill essential information, and interact effectively across a range of complex tasks, underscoring their transformative impact on NLP.

Despite these strengths, LLMs exhibit distinct limitations. As closed systems, they are unable to access up-to-date information or specialized domain knowledge in real-time, which can result in errors or 'hallucinations' [4] where they provide incorrect or fabricated information that appears plausible. Hallucinations occur because LLMs function by predicting the next most probable sequence of words based on patterns learned from their training data, without an inherent understanding of factual correctness or logical coherence. This process, rooted in probabilistic modeling, compels them to produce fluent responses that align with linguistic context rather than verifiable truth. The issue is exacerbated when the model encounters gaps, inconsistencies, or biases in the training data, leading it to "fill in" these gaps with plausible but incorrect details. Furthermore, the confidence with which hallucinations are presented stems from the LLM's optimization for linguistic fluency and relevance, rather than a validation mechanism to cross-check factual claims, resulting in outputs that can mislead users despite their authoritative tone. Fine-tuning refers to the process of adapting a pre-trained LLM to specific tasks or domains by updating its neural network parameters using task-specific or domain-specific data. While this approach allows the LLM to learn nuances and details relevant to a particular area, it often comes at the cost of generality. For instance, while fine-tuning offers a potential solution to address these issues, it can reduce the LLM's versatility [5] [6] by requiring adjustments to parameters specific to each area of expertise. Further challenges, such as performing tasks that require complex multi-step reasoning or staying updated with evolving knowledge, remain substantial.

These limitations highlight the need to integrate external resources and specialized tools, opening pathways for the development of autonomous agents that collaborate with LLMs to expand their capabilities. By incorporating external tools, LLMs can overcome the inherent constraints of closed systems, such as their inability to access real-time or highly specialized domain information. Retrieval-Augmented Generation (RAG) systems exemplify this approach, allowing LLMs to query and incorporate external knowledge sources to improve their accuracy, adaptability, and relevance [7]. For example, while LLMs can generate fluent text, they cannot independently provide real-time data, such as current weather updates, or interact directly with databases and APIs [8]. This limitation underscores the critical role of tool use in extending the utility of LLMs beyond their pre-trained knowledge. Tool use not only addresses knowledge gaps but is also key to the development of autonomous systems - where the LLM, as opposed to a hard coded set of rules, drives the system's actions - capable of logical reasoning, querying, and executing tasks in dynamic environments. Thus, tool-assisted LLMs are becoming a focal point in research, aiming to optimize both the retrieval of relevant tools and their effective utilization to solve complex problems. In this work, we explore methods to enhance tool-assisted LLMs, with a particular emphasis on maximizing the efficiency and accuracy of tool retrieval, thereby addressing real-world challenges more effectively.

1.1. Motivation

This work aims to explore if Query Decomposition, Reformulation, and Self-refine have a meaningful, positive impact on an autonomous, multi-agent RAG system which leverages function calling to shortlist tools and APIs designed to solve user queries that require external, specialized knowledge, and then uses those tools to solve the queries and present their solutions.

The premise behind query decomposition and reformulation is that reframing the original user query — by reducing redundant text and breaking it into ordered sub-components — may enhance the retriever's ability to find the most suitable APIs, and the solver to help plan what order to call the tools in. This approach can be particularly valuable for complex queries with dependencies, where solving one component is necessary before addressing another. The principle of Self-refine aims to enable the system to make iterative improvements by having the generative LLM review its own output, identify possible causes in case of failure, and incorporate those for improvements in subsequent iterations of solving the query.

The motivation for focusing on autonomous agents stems from our goal of ensuring the system remains generalizable across different domains. In this context, autonomous agents leverage the reasoning capabilities of LLMs to plan and determine subsequent actions. This approach is expected to scale more effectively than explicit rule-based programming, particularly when addressing circular logic or unforeseen situations and domains. We build upon the principles of AnyTool, which employs a hierarchical agent system to eliminate the costly step of training a separate retriever — which typically involving extensive dataset generation, solution path annotations, and model training.

1.2. Research Questions

We aim to address key research questions in the field of tool-assisted question answering (QA) with LLMs. By answering the three research questions, we seek to identify and compare the advantages of Query Decomposition, Self-refine, and Reformulation in an autonomous agent-based system without an explicitly trained API Retriever.

The outcomes of these studies aim to provide further insight into methods of best maximizing the performance of black box Tool-selection LLMs which are typically closed source and have parameters which cannot be modified. The research questions are outlined here.

1. Does Query Decomposition enhance the retriever system’s ability to shortlist appropriate tools and solve queries?

This question investigates whether simplifying a query by breaking it as a sequence of ordered steps improves the system’s problem-solving accuracy compared to directly processing the original query in its initial form.

2. Does Self-refine add meaningful value to the reflection pipeline of a tool-use RAG system after an error in the first round of solving?

Here, we explore whether a tool-selection LLM system can better identify and correct errors when the solver model reviews its own provisional answer to produce feedback, and the retriever is then fed the incorrect answer and feedback on why it failed, compared to cases where only feedback from an unspecified, possibly different evaluator model is provided.

3. Does reformulating the query and starting afresh during reflection improve tool retrieval quality and problem-solving ability?

This question examines the impact of query reformulation on retrieval and solving effectiveness, comparing the standard approach — where the LLM uses its entire conversation history — against an approach that involves rewriting the initial query based on first-round feedback and starting with a fresh conversation context.

Our results demonstrate that our new, proposed system - inclusive of the Decomposer, Self-refine and Reformulator Agent - is better than the baseline at solving multi-part queries that require 3 or more tools to solve. This result is significant at a 5% significance level. We also observe a general improvement in tool retrieval and query solving across simpler queries based on our experimental results, but our dataset is too limited to conclude that the improvement is statistically significant. Ablation studies show that the most crucial new component is the Query Reformulator Agent, followed by the Query Decomposer, while Self-refine has a negligible effect.

2. Fundamentals

2.1. Question Answering

In the context of LLMs, Question Answering (QA) refers to the process by which an LLM generates responses to user queries based on patterns learned during training. Models such as OpenAI’s GPT and Meta’s LLaMA are trained on extensive datasets, enabling them to address a wide variety of questions. These range from straightforward factual inquiries to more complex questions that involve arithmetic, logical reasoning, or interpretative analysis. By leveraging linguistic and contextual patterns derived from vast textual corpora, LLMs can produce answers relevant to general knowledge, technical topics, and even subjective areas such as advice or opinion.

However, the QA capabilities of LLMs are subject to inherent limitations. Because LLMs are bound by the data they were trained on, they are unable to access or generate responses involving recent developments, specific domain expertise, or technical details that lie outside their training data. In such cases, their answers may be inaccurate, outdated, or even entirely fabricated — what is commonly referred to as a “hallucination” [4]. This limitation arises because the LLMs generate text by predicting the most likely sequence of words based on the patterns they learned during training. They lack an inherent ability to assess the factual accuracy or logical validity of their responses. This probabilistic approach drives them to focus on linguistic fluency over factual correctness. When the training data contains inadequacies, the LLM compensates by generating believable but incorrect text, with a confident tone. LLMs do not “understand” content as a human does; instead, they produce responses based on probabilistic associations within the text data they have processed. For example, if asked about a recently published research paper, an LLM may provide an irrelevant or erroneous answer due to its inability to access current information.

To mitigate this issue and enhance the QA performance of LLMs, researchers have developed techniques such as Retrieval-augmented Generation (RAG). In RAG, the LLM integrates external knowledge sources through a retriever component that, when provided a prompt and user input, identifies the most relevant documents or information to support the LLM’s response. The retrieved information is used by the reader or

solver for in-context learning, where it incorporates the external data directly into its prompt. This approach significantly improves the system's ability to handle specialized or current queries, reducing hallucinations and increasing answer accuracy. Empirical studies observe that RAG systems outperform even unsupervised fine-tuned LLMs on knowledge-intensive tasks [9]. By incorporating external sources, RAG mitigates the limitations of standalone LLMs, bridging the gap between the LLM's static knowledge and the need for contextually relevant responses.

2.2. Tool Use

Tool use, or tool selection, refers to the ability of LLMs to not only answer queries based on its training, but also identify and interact with the appropriate external tools to solve more complex tasks [10]. While traditional LLMs rely on static knowledge encoded in their parameters during training, they often encounter limitations when dealing with tasks requiring specific expertise or up-to-date information, and susceptible to hallucinating incorrect information. This is where tool assistance becomes important, as it enables the LLM to access external tools, databases, or APIs to complement its internal knowledge. For example, an LLM may use a calculator to solve arithmetic problems, retrieve recent data from a database, or interact with domain-specific applications to perform specialized tasks.

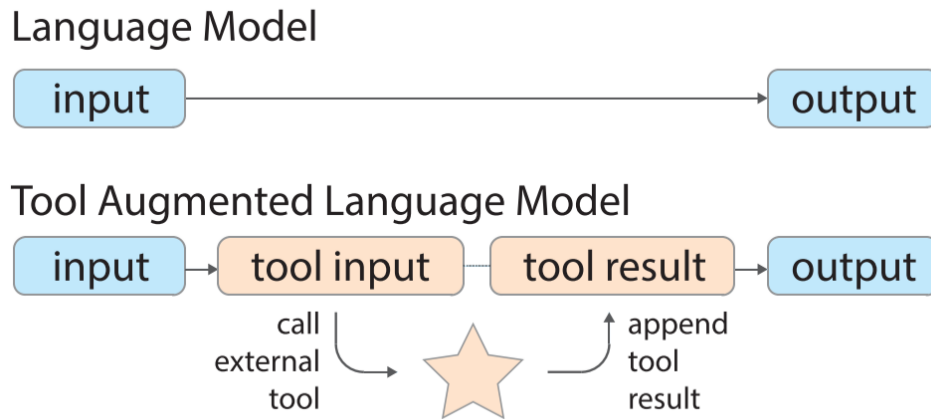


Figure 2.1.: LLMs can use external tools to enhance their capabilities [11]

Tool selection extends the practical capabilities of LLMs by addressing their inherent limitations. In such scenarios, the LLM must analyze the user's query to identify the

most relevant external tools and then effectively employ them to produce a response. This process often necessitates multi-step reasoning, where the LLM first breaks down the problem into sub-tasks, determines which of these sub-tasks can be solved internally, and which ones require the use of an external tool. In doing so, the LLM transitions from text generation to performing real-world tasks efficiently across domains where precision or specialized knowledge is essential.

2.3. Autonomous Agents

Current research in AI is increasingly opting for multi-agent systems over single LLMs due to limitations inherent in single LLM frameworks.

- **Specialized Expertise:** Multi-agent systems allow each agent to focus on specific sub-tasks, enabling more accurate and context-aware solutions than a single LLM can provide.
- **Improved Error Handling:** Agents can collaborate to detect, correct, and refine outputs, improving reliability for complex problem-solving.
- **Load Balancing:** Dividing tasks among agents helps manage token and context limitations, making it easier to handle large, complex information.

An autonomous agent system powered by an LLM utilizes the reasoning capabilities of the LLM as its central decision-making mechanism, incorporating components like **planning**, **memory**, and **external tools** to manage complex tasks effectively. These agents decompose tasks into sub-goals, reflect on prior actions, and access external information, all of which contribute to more effective problem-solving. [12]

Planning is the primary function within an autonomous agent's operation. Through planning, the agent divides complex tasks into a sequence of manageable sub-goals, allowing it to approach intricate problems incrementally and simplifying the path to solutions. A crucial facet of this process is the agent's capacity for reflection and refinement. After completing a task or sub-goal, the agent performs self-evaluation, assessing past actions and outcomes—a process referred to as Reflection and detailed in the next subsection. This reflective process, part of the Planner's function, enables the agent to learn from past actions, thereby improving its future performance and refining strategies to achieve optimal results.

Memory in LLM agents is divided into short-term and long-term components. Short-term memory is represented by in-context learning, where the model temporarily

retains information within a limited history window during a session, focusing more acutely on recent interactions. Long-term memory, on the other hand, is typically managed through external mechanisms such as vector stores and retrieval systems, enabling the agent to retain and access substantial volumes of data over extended periods. This approach extends the agent’s recall abilities far beyond its immediate processing capacity, allowing it to retrieve relevant information as required.

Tool Use, as outlined in the preceding section, enhances the agent’s performance by facilitating interaction with external resources. With integrated tool selection, autonomous agents can leverage their robust planning capabilities to dynamically balance between internal knowledge and external resources, thereby executing tasks more efficiently and accurately.

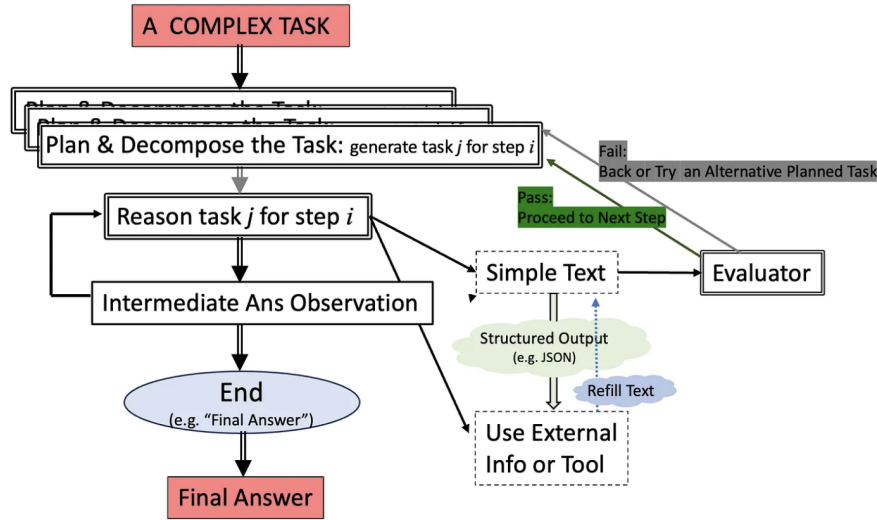


Figure 2.2.: Workflow of an autonomous agent [13]

2.4. Reflection

In LLM prompting strategies, Reflection refers to the model’s capability to evaluate and refine its responses by critically examining the reasoning and accuracy of its initial outputs [14]. This iterative process enables the model to self-assess and make strategic modifications, establishing a feedback loop that supports the generation of more reli-

able and contextually relevant responses. After producing a preliminary answer, the LLM evaluates its adequacy in addressing the input query, identifies potential logical inconsistencies, and considers areas for improvement, iterating as needed to optimize quality and relevance. [15]

The principle of Reflection is essential for mitigating limitations such as hallucinations, where the LLM might produce erroneous or fictitious information. By embedding Reflection into its operational flow, the LLM can more effectively ensure factual accuracy and align more closely with user intent. This is especially beneficial in tasks requiring high precision, including multi-step problem-solving, autonomous tool use, or information retrieval. Reflection fosters coherence across complex tasks, providing an internal quality-assurance mechanism that contributes to a more dependable final output.

2.5. Query Rewriting

Query Rewriting involves transforming an initial input query into a rephrased or restructured form with the objective of improving retrieval accuracy or problem-solving effectiveness [16][17]. Common applications of query rewriting are seen in search engines, through auto-correcting typos, phrase substitution (e.g., converting “Celsius” to “Fahrenheit” for U.S.-based searches), or enabling semantic search [18] [19]. These methods typically involve minimal intervention, with adjustments limited to basic adjustments. However, Query Rewriting can also involve more substantial modifications, refining the original queries by removing redundant details or decomposing complex questions into a series of structured sub-questions.

$$Q_1 = \text{Rewriter}(Q_0, \dots)$$

where Q_0 is the initial query, Q_1 is the rewritten query, and “...” represents any additional information provided to the Rewriter [16][20].

Query Decomposition and **Query Reformulation** are essential strategies within the broader scope of Query Rewriting, aimed at maximizing the performance of LLMs in QA and Knowledge Retrieval tasks. By applying these techniques, the LLM can more effectively address complex or ambiguous queries, leading to a more accurate and targeted final response.

2.5.1. Decomposition

Query Decomposition refers to the process of breaking down a complex query into smaller, manageable sub-queries, which are easier for the LLM to process. By addressing the larger query through sub-queries, this method enables the LLM to provide more accurate and comprehensive responses to complex questions.

$$Q_d = \text{Decomposer}(Q_0)$$

where Q_0 is the initial query, Q_d is the rewritten query, and no additional information is provided to the Decomposer

Query Decomposition can help standardize the structure of the query received by an agent-system through a template of what information a good query is expected to provide. This in turn can help improve the decision-making of the solver by providing it with a degree of hand-holding on how to break down problems.

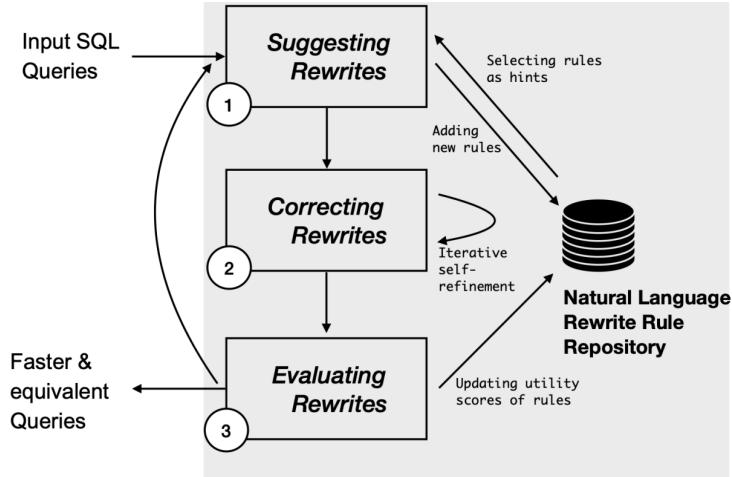


Figure 2.3.: Generic workflow of Query Rewriting [21]

Given a multi-part question like, "Who is the all time top scorer of the team that won the 2008 UEFA Champions League?" the LLM should first find which team won the 2008 UEFA Champions League, then move on to finding the list of top scorers for that specific team. This expected sequence of breaking down a query can be made likelier

by rewording the query to something equivalent to "First find the name of the team that won the 2008 Champions League, and then find the name of the all time top scorer of that team."

2.5.2. Reformulation

When the system uses reflection to correct errors from an initial attempt at solving the query, Query Reformulation involves rephrasing or modifying the query based on the initial results provisionally provided by the LLM. If a part of the multi-part query has been addressed in the first pass, then it may be excluded from the new query sent to the retriever or solver. Additionally, if there is some feedback that caused the solver to fail in the first attempt, that too may be included to guide the system better in the second pass.

$$Q_r = \text{Reformulator}(Q_0, A_0, Fb_0)$$

where Q_0 is the initial query, A_0 is the initial answer, and Fb_0 represents feedback. [20]

This iterative process allows the system to refine its responses over time, ensuring better accuracy and relevance.

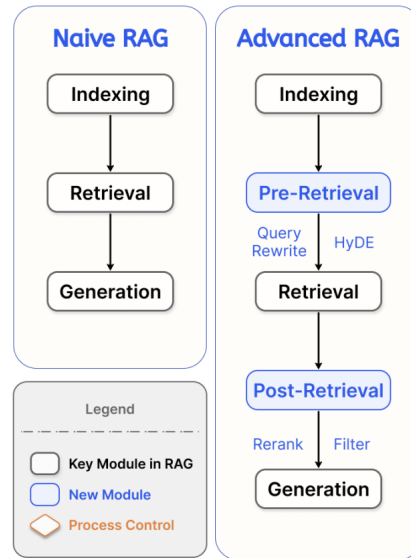


Figure 2.4.: Our system goes beyond naive RAGs for the rewrite to aid retrieval [22]

3. Related Work

This chapter reviews literature on the key topics integral to our research and covered in the previous chapter: Reflection, Query Rewriting, Tool Use, and Autonomous Agents.

3.1. Reflection

In Knowledge Retrieval or RAG systems, where the final output quality depends on retrieval accuracy, feedback provided by the LLM to the retriever—or to itself—after an inadequate solution can guide the model towards a more accurate answer in subsequent iterations. This section examines existing studies on iterative improvements in model outputs.

3.1.1. LLattribution

An influential work focused on improving document selection within retriever-assisted Knowledge Retrieval systems is LLattribution by Li et al. [23], which enhances retrieval by enabling the LLM solver to provide feedback on the relevance of retrieved documents. Before generating an answer, the LLM evaluates whether the provided documents are sufficient for the query, producing a binary decision. If the documents are found inadequate for answering the query, the LLM delivers feedback, identifying missing information, and recommending specific documents for further examination. This iterative process continues until the retrieved documents are satisfactory.

Retrofit Attribution using Research and Revision (RARR) by Gao et al. [24] is a model-agnostic approach designed to improve LLM attribution. It operates by first generating a draft answer, then formulating sub-queries to verify the draft’s accuracy. Evidence is retrieved for each sub-query, and an agreement model assesses the alignment between the evidence and the draft, identifying any contradictions or consensus. Based on its feedback, revisions are made to the draft, improving its accuracy without restricting the initial generation. This approach exemplifies a Generation-Augmented Retrieval method, where retrieval is dynamically guided by the model’s inherent knowledge and reasoning.

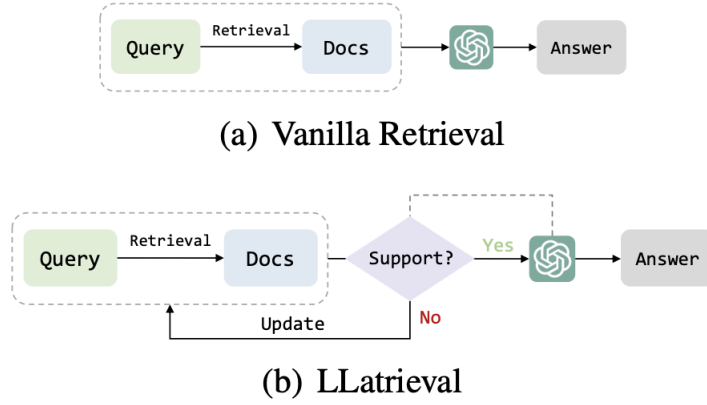


Figure 3.1.: LLaTrieval uses the LLM to judge when retrieval is complete [23]

RePlug introduced by Shi et al. [25] aims to improve the response quality of black-box LLMs by refining document relevance. Initially, a standard retriever selects a set of N documents related to the input query. Each document is then prepended as context and inputted to the LLM individually. A weighted average of potential outputs is calculated, where weights are derived from the softmax of the cosine similarity between each document and the query, effectively optimizing the final response based on document relevance. Similarly, the **In-Context Retriever-Augmented Language Model (RALM)** by Ram et al. [26] applies this approach but leverages off-the-shelf retrievers that require no additional training.

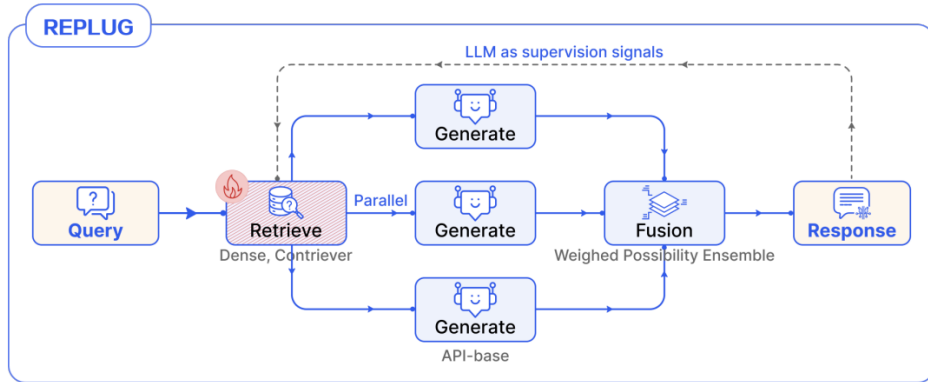


Figure 3.2.: RePlug composes an answer as a weighted average of each individual retrieved document [25] [22]

3.1.2. Self-Refine

Self-refine, introduced by Madaan et al. [27], presents an iterative prompting technique to improve the quality and precision of LLM outputs through structured self-reflection. This method encourages the LLM to analyze and refine its own responses based on feedback from previous, potentially erroneous, attempts. The process involves the LLM generating an initial response, evaluating this response for accuracy and coherence, and then using self-generated feedback to make necessary adjustments in a refined output.

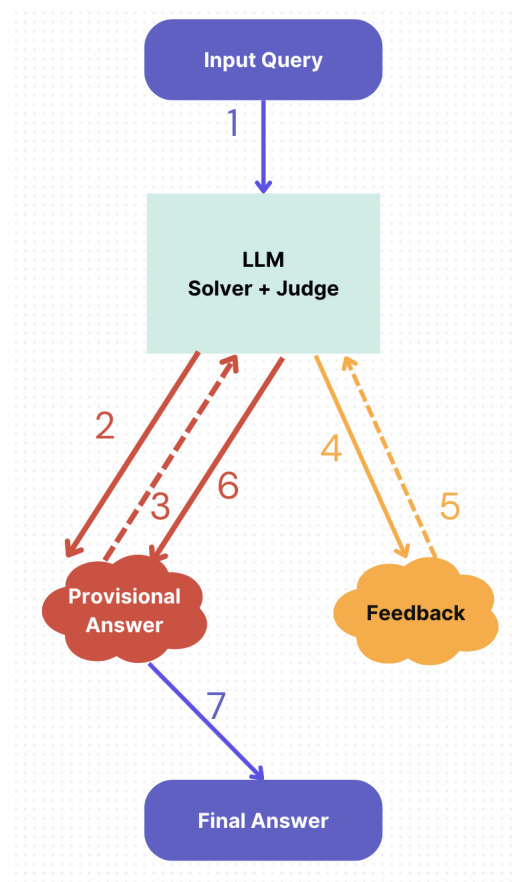


Figure 3.3.: Self-refine sees the LLM review its own interim output and uses that feedback to generate a new answer

Unlike approaches where feedback is provided directly by a human or an external

system, Self-refine harnesses the LLM’s own reasoning capabilities to identify errors and guide its subsequent responses. This method is distinct in that the model does not merely respond to external feedback; instead, it proactively engages in a self-correcting loop, using intermediate insights to improve its final response—a process resembling the self-editing and iterative refinement strategies employed by humans. Unlike LLattribution, RARR, or RePlug, it is not designed specifically for Retrieval-based systems, but a general principle for any LLM.

3.2. Query Rewriting

Query Rewriting is the process of modifying the original input query to enhance retrieval accuracy and problem-solving efficacy. Some frameworks, such as **Iter-RetGen** by Shao et al. [28], approach query rewriting as a post-retrieval step. Here, the “rewritten query” is essentially a concatenation of the initial query with a provisional answer, acting as a form of iterative reflection. Similarly, **Query2doc** by Wang et al. [29] generates pseudo-documents post-retrieval to append to the original query to improve response relevance. ConvGQR by Mo et al. [30] reformulates conversational queries based on pre-trained LLMs, building on research by Vakulenko et al. [17]

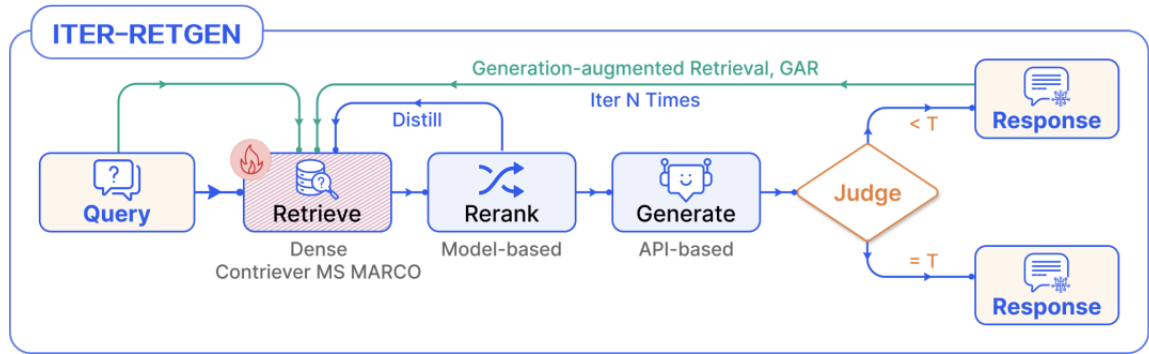


Figure 3.4.: Iter-Retgen "rewrites" the query to guide retrieval with a blind guess.[28]
[22]

For our purposes, we focus on query rewrites performed before the retrieval process to ensure the retriever benefits from the decomposition too. **GenQR** by Wang et al. [20], introduces a zero-shot approach for query reformulation prior to retrieval. This method employs a natural language instruction prompt that outlines the desired characteristics of the rewritten query. Both the prompt and the initial query are fed into an LLM to

generate a refined query, often by adding keywords or simplifying the phrasing to enhance retrieval effectiveness.

GenQREnsemble by Dhole et al. [31] extends this very concept with an ensemble approach to avoid “overfitting” to a single reformulation. This technique uses N distinct but semantically similar prompts to generate keywords from the LLM. Keywords produced from each prompt are combined with the original query to form the rewritten query, which is subsequently sent to the retriever or solver, thus diversifying the retrieval approach to improve accuracy.

As conversations extend over many rounds, LLM performance can degrade due to limitations in retaining and prioritizing long-term context, as noted by **Maharana et al.** [32] who evaluated long-term conversational memory of LLM agents. In dialogues with numerous turns - as is common in our application of tool-use for QA tasks - LLMs may struggle to maintain coherence and relevance without a structured memory system capable of tracking evolving conversational elements. Processing extensive conversational histories and context can lead to issues such as “forgetting” earlier parts of the conversation or diluting attention on recent interactions. Without mechanisms for compressing, retrieving, or prioritizing essential information, model responses may include irrelevant past details or misalign with the current topic. This serves as inspiration for Query Reformulation in the reflection pipeline of our system.

3.3. Tool Use

Research in Tool-augmented LLMs (TALMs) has aimed to enhance problem-solving capabilities by integrating various different principles, such as including curated examples from task libraries, training solvers with solution-path annotations, proxy placeholder values, or function-calling mechanisms.

Automated Reasoning and Tool-Use (ART) by Paranjape et al. [33] is a framework designed to augment LLM capabilities through multi-step reasoning and external tool usage during task completion. ART dynamically selects relevant examples from a pre-curated task library, demonstrating appropriate tool choices and reasoning strategies for new tasks, without requiring additional model training. The generation process halts whenever a tool is needed, integrates the tool output, and then resumes.

ART uses a structured query system to govern each reasoning step and tool interaction. Users can update task and tool libraries with new examples, allowing the framework to remain adaptable to evolving requirements and tools.

Toolformer by Schick et al. [34] addresses the limitations of traditional LLMs — such as accessing real-time data, performing precise calculations, and minimizing hallucinations — by selectively utilizing external tools or APIs. Unlike standard LLMs confined to internal knowledge, Toolformer learns to detect when external assistance is beneficial, select an appropriate tool, and execute it. Training Toolformer involves three main steps: **(1)** Sampling potential API calls during training, **(2)** Executing these calls, and **(3)** Filtering out irrelevant results. This process enables Toolformer to improve accuracy and dependability for tasks such as real-time data retrieval and complex calculations, thereby expanding its practical applications.

Chain of Abstraction by Gao et al. [35] provides a flexible approach to tool selection in reasoning tasks requiring interconnected tool calls. Reasoning chains are initially structured with placeholders, which the system subsequently fills with precise information obtained from external tools. This approach enables general-purpose reasoning strategies that integrate specialized tools, enhancing its effectiveness in multi-step tasks.

3.3.1. ToolLLM

ToolLLM by Qin et al. [36] represents a notable advancement in tool-use frameworks by integrating two primary components: an API Retriever that recommends relevant APIs based on an input query, and a fine-tuned LLaMA-2 7B model, similar to Gorilla developed by Patil et al. [37], trained on annotated solution paths, enabling it to solve queries through iterative API calls.

API Retriever

The API Retriever component is trained on pairs of queries and corresponding lists of ground truth APIs necessary for solving them. To generate these pairs, an OpenAI model is given a catalog of APIs and prompted to produce instructions or queries that utilize a subset of these APIs. The instructions are categorized into three levels of API usage: intra-tool usage (G1), multi-tool usage within the same category (G2), and multi-tool usage across different categories (G3). Sentence-BERT is employed to embed these instructions and API lists, enabling a dense retriever to efficiently match queries with relevant APIs.

Solver

Once the Retriever identifies a list of APIs, the Solver engages in multi-step reasoning to generate an annotated solution path. Function versions of the selected APIs are created based on their documentation, specifying required parameters, optional parameters,

and default values. These functional APIs are then integrated into the Solver, which employs the **Depth-First Search Decision Tree (DFSDT)** algorithm for structured reasoning.

The DFSDT algorithm addresses limitations in **Chain-of-Thought (CoT)** [38] and **ReAct** [39] methodologies. CoT prompting guides an LLM to reason step-by-step by explicitly breaking down complex problems into smaller, logical steps, improving reasoning accuracy. ReAct combines reasoning and action by prompting the LLM to complement its thought process with actions, enabling it to reason about tasks while interacting with tools or environments. However, CoT and ReAct suffer from two main issues: error propagation—where early missteps lead the model astray, resulting in errors that accumulate with no corrective mechanism—and limited solution exploration, as they follow a single reasoning path without flexibility. DFSDT overcomes these issues by exploring multiple reasoning paths and allowing the system to pivot away from unproductive directions. It incorporates backtracking, using a signaling function that indicates when it is advisable to abandon the current approach in favor of alternative solutions. This branching approach expands the search space, offering the model more pathways to arrive at a correct solution. This dataset of queries and the corresponding annotated solution path - ToolBench - is used to fine-tune the LLaMa-2 7B model [3] to obtain the solver model, ToolLLaMa.

To facilitate this process, additional functions are defined to allow the model to *finish* a reasoning trajectory, providing an answer when one is found or *abandon* a path if it leads to a dead end. These function calls and their outcomes are recorded in the conversation history, allowing the model to incorporate and review each thought-call pair iteratively. This iterative, branching method enables the model to produce a sequence of thought-call pairs, with each API call followed by a reflective thought based on the resulting state change.

The ToolLLM system utilizes tools and APIs from RapidAPI, which offers over 16,000 APIs across more than 3,000 tools in 49 categories such as Sports, Entertainment, Finance, and Weather.

When the Solver determines that it has generated a viable answer, it submits the answer along with the original query to an Evaluator model. This model classifies the attempt as “Passed” if the answer satisfactorily addresses the query or “Failed” if it does not. In cases marked as “Failed,” the Evaluator provides feedback on the attempt’s shortcomings, guiding the Solver in potential refinements.

3.4. LLM-powered Autonomous Agents

Multi-agent systems employ LLMs as autonomous agents that collaborate, communicate, and negotiate to accomplish complex tasks. Each *agent* in such a system operates as an instance of an LLM, potentially with specific role-based specializations such as data preprocessing, inference, or decision-making.

By distributing tasks among agents, these systems achieve improved consistency and efficiency through predefined protocols that dictate the functionality and hierarchy of each agent, allowing structured, role-specific contributions.

A notable application of this concept is **Multi-agent Review Generation (MARG)** by D’Arcy et al. [40]. MARG employs a tiered system of agents to generate scientific paper reviews, organizing agents across three specialized roles: a Leader to coordinate inter-agent communication, Workers to process designated text sections, and an Expert focused on specialized review aspects. The review process is further divided into three parallel multi-agent systems, each focused on distinct paper aspects—Experiments, Clarity, and Impact. Each system comprises its own Leader, Workers, and Expert. The outputs of these parallel systems are then consolidated and refined by a final multi-agent system, *Refinement*, which synthesizes a comprehensive review. While this architecture efficiently narrows the scope of each agent’s responsibilities, focusing their efforts to maximize processing precision, it does not yet integrate tool-use capabilities that would enable external interactions for enhanced operations.

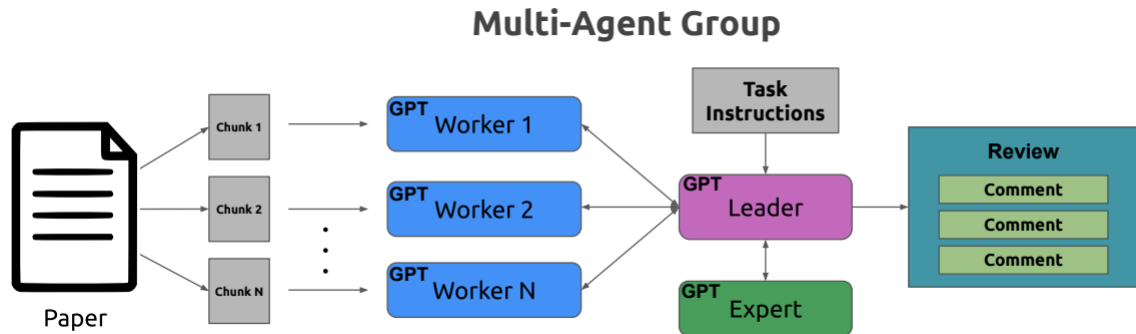


Figure 3.5.: MARG [40] uses a multi-agent architecture where agents have different roles to combine for the task of reviewing a scientific paper

3.4.1. Function Calling

OpenAI's recent addition of Function Calling [41] equips LLMs with the capacity to invoke specific functions autonomously during conversations, enhancing their reasoning and decision-making abilities for more interactive and goal-oriented responses. Rather than relying solely on natural language to convey information, GPT models (versions 3.5 and above) can dynamically determine when to call these functions, allowing them to retrieve, process, or structure information in task-specific ways. Key aspects of Function Calling include:

- 1. API Definitions:** Users can define functions with a structured JSON format that includes fields such as function name, description, required parameters, and data types. This setup enables GPT to interface with external APIs, access databases, or carry out designated actions effectively.
- 2. Autonomous Calls:** GPT can autonomously decide to call a function based on the context of the input text, facilitating tool use without explicit rule-based programming. This dynamic capability is especially useful in applications that require real-time data retrieval or processing.
- 3. Function Arguments:** When initiating a function call, the LLM can extract relevant arguments from the input to populate the function's parameters. The function's output is then returned to the LLM, which integrates the result into the conversation seamlessly, providing responses enriched with the real-time information or task-specific results.

Listing 3.1: Sample list of functions for input text related to vehicles

```
1 motor_functions = [  
2     {  
3         'name': 'extract_car_info',  
4         'description': 'Get the information about the car model from the  
5             text',  
6         'parameters': {  
7             'type': 'object',  
8             'properties': {  
9                 'name': {  
10                    'type': 'string',  
11                    'description': 'Name of the car manufacturer and model'  
12                },  
13                'top_speed': {  
14                    'type': 'integer',  
15                    'description': 'Maximum speed of car in kmph'  
16                },  
17            },  
18        },  
19     ],
```

```
16         'time_to_hundred': {
17             'type': 'float',
18             'description': 'Time to go from 0 to 100 kmph'
19         }
20     }
21 }
22 },
23 {
24     'name': 'extract_bike_info',
25     'description': 'Get the information about the bike from the text',
26     'parameters': {
27         'type': 'object',
28         'properties': {
29             'name': {
30                 'type': 'string',
31                 'description': 'Name of the bike model and manufacturer.'
32             },
33             'mileage': {
34                 'type': 'integer',
35                 'description': 'Distance travelled per unit of fuel'
36             }
37         },
38         'required': ['name']
39     }
40 }
41 ]
```

3.4.2. AnyTool

Previous frameworks like Toolformer and ToolLLM have required extensive training of retrievers and solvers to enable tool-use functionality. AnyTool by Du et al. [42] innovates by eliminating the need for such external module training, instead relying solely on the Function Calling approach. This system comprises three core components: a hierarchical API retrieval system, a solver, and a self-reflection pipeline.

API Retriever

The API retrieval process in AnyTool is structured with three hierarchical agents — Meta, Category, and Tool—to efficiently narrow down relevant APIs.

1. Meta Agent: Upon receiving the initial user query, the Meta agent uses a System prompt to broadly assess the most relevant API categories. This agent has access

to all categories and the tools within them and serves as a high-level decision-maker to initialize the search.

2. Category Agents: For each identified category, the Meta agent creates Category agents, each of which has access to tools and APIs specific to its category. These agents further refine the search by identifying tools within their category that best align with the query.

3. Tool Agents: Once suitable tools are identified, Tool agents are instantiated to select specific APIs from each tool. These agents populate a global pool of shortlisted APIs with options most relevant to solving the query.

Instead of relying on popular commercial frameworks such as LangChain [43] or CrewAI [44], the properties of these agents, such as functions, variables, prompts etc. are designed from scratch.

The API retrieval is dynamic and recursive, utilizing principles from **LLattribution** [23]. After each API addition, a planning LLM assesses whether the query can be addressed using the accumulated pool. If deemed Solvable, the retrieval process halts, and the query is passed to the Solver LLM along with the selected APIs. If labeled Unsolvable (for instance, due to missing or invalid data), retrieval continues with more Category and Tool agents searching for additional solutions. Next, the Solver model receive the query and API list for final resolution.

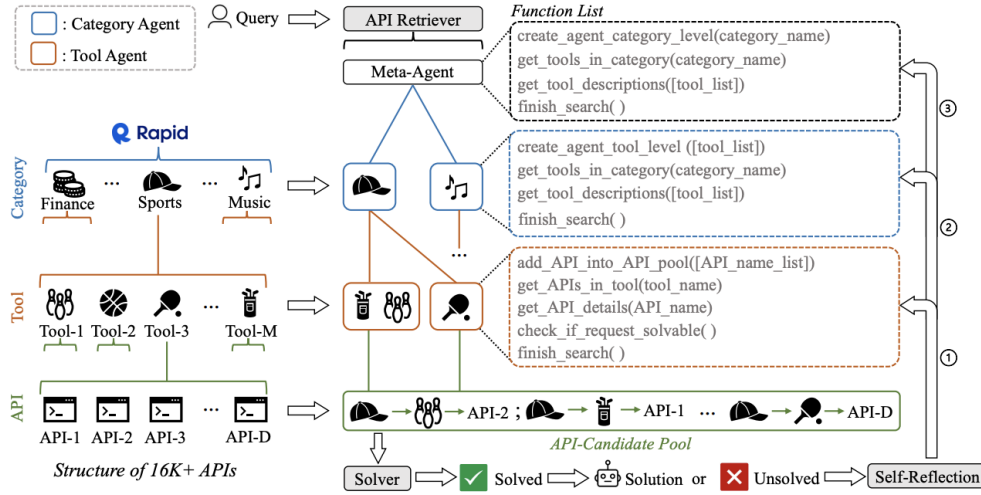


Figure 3.6.: Workflow of AnyTool with a hierarchical API retriever to sift through 16,000+ APIs [42]

Solver

Once the relevant APIs are shortlisted, they undergo preprocessing to convert them into callable functions with necessary attributes such as required parameters, optional parameters, and default values, as outlined in the tool documentation. These functions are then integrated with the Solver, which utilizes the Depth-First Search Decision Tree (DFSDT) algorithm originally introduced in ToolLLM.

Using the DFSDT algorithm along with these callable functions, the model is prompted to produce diverse *thought-call* pairs. In each pair, a function from the list is called, and each subsequent thought is generated by integrating the results of any changes produced by the function’s output.

If the Solver generates an answer, it forwards this answer along with the original input query (from before decomposition) to the evaluator model. The evaluator then classifies the attempt as “Passed” if the answer is deemed acceptable for the query or “Failed” if it is unsatisfactory. For failed attempts, the evaluator provides feedback explaining why the solution was inadequate.

Self-Reflection

When the evaluator deems an answer to have “Failed,” AnyTool’s reflection mechanism is re-engaged. During this process, agents from the retrieval hierarchy (Tool, Category, and Meta) receive the evaluator’s feedback and assess potential next steps.

The agents re-activate sequentially, from the lowest level (Tool agents) to the highest (Meta agent).

- 1. Tool Agents:** These agents first determine whether any alternative APIs within their respective tools could potentially solve the query. If suitable APIs are identified, they are added to the global API pool; otherwise, the Tool agents are closed.
- 2. Category Agents:** If Tool agents cannot identify a solution, Category agents assess if any other tools within the same category might address the query. If promising tools are found, new Tool agents are created to identify relevant APIs and add them to the pool; otherwise, the Category agents are closed.
- 3. Meta Agents:** If neither Tool nor Category agents succeed, the Meta agent considers if a new category of APIs might be necessary. If so, it creates new Category agents to search within that category; if not, the Meta agent halts, and the system discontinues efforts on the query.

This structured reflection hierarchy provides a systematic fallback mechanism, enhancing AnyTool’s ability to explore alternative solutions based on evaluator feedback before ultimately giving up if no solution is found.

4. Methodology

This chapter outlines the key features introduced in our system, which builds upon principles from previous works like AnyTool and ToolLLM. Our objective is to explore tool use with autonomous LLM agents, particularly leveraging black-box models and minimizing the need to train components such as the retriever or solver. To achieve this, we extensively rely on the function calling capability of OpenAI models.

4.1. Query Decomposition

One of the key modifications we introduce to AnyTool is the implementation of Query Decomposition, which occurs as the first step upon receiving the original input query. In the original AnyTool framework, the Meta agent handles API retrieval by first identifying the appropriate tool categories. In our system, however, a Query Decomposer pipeline is inserted before the Meta agent receives the input query. The Query Decomposer uses an LLM in conjunction with a System prompt that provides specific instructions on how to rephrase the query. These guidelines focus on eliminating irrelevant details (e.g., salutations), simplifying language, and rewording the query into a structured format that can be better understood by the retriever and solver. Along with instructions, the decomposition prompt includes few-shot examples that demonstrate how to break down complex queries, a topic explored by Yu et al. [16]. In cases where a query contains multiple dependent parts, the Decomposer reframes the query into a sequence of action steps, guiding the retriever and solver through the process.

It is important to note that the application of decomposition here **does not** equate to the original query with N dependent components being broken down into a chain of N individual and separate queries. From observed experience, that extreme is deemed unnecessarily costly when the middle ground solution - a single query with simpler language of ordered action steps - is found to be sufficient to guide both the retriever system and the solver LLM.

Inspired by GenQREnsemble [31], we aim to avoid overfitting or excessive corrections in the decomposition. Rather than an ensemble of LLM calls with similar prompts, we have the Decomposer generate a single reformulated query. The LLM then provides a

boolean response indicating whether the decomposition accurately reflects the intent of the original input query. If the response is *True*, the decomposed query becomes the new input for the retriever and solver. If the response is *False*, the original query is retained. If the decomposition is valid, all agents in the retriever and solver work only with the new, decomposed query throughout the process.

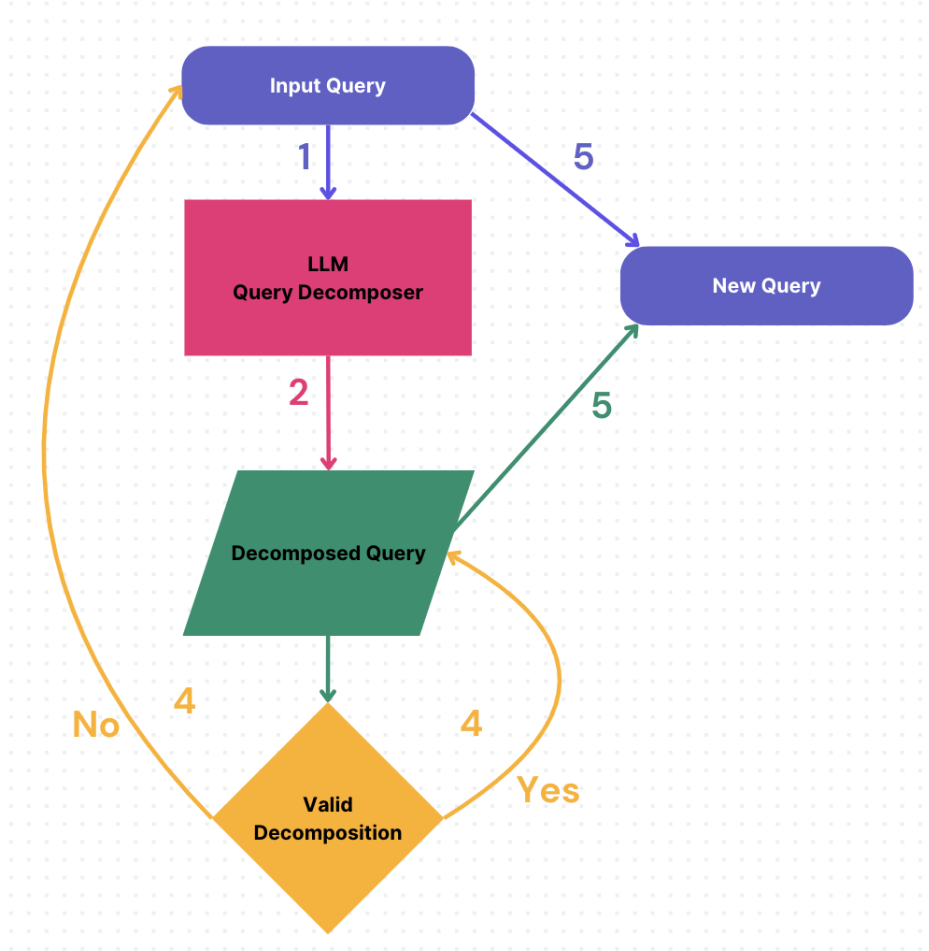


Figure 4.1.: Workflow of the Query Decomposer

However, to ensure that the system is evaluated and benchmarked on user intent, which is captured in the original input query, the evaluator still compares the generated answer with the user-provided input query to judge if the solution produced is accurate - i.e if the system has "*passed*".

4.1.1. System Prompt for Query Decomposer

```
You are QueryDecomposerGPT, designed to analyze and break down complex queries.
Your job is to clarify the logical order of query components, especially when
dependencies exist, and produce a decomposed version following these rules:
1. For dependent parts, decompose step by step, specifying the order.
Example: "What is the capital city of the second most populated state of the United
States?" becomes "Find the most populated US state, then its capital city."
Example: "Who is the longest-serving active Head of State in Euro-using countries?"
becomes "List Euro-using countries, their current Heads of State, then identify
the longest-serving one."
2. For independent parts, retain the query as is.
Example: "Get the Spotify top 20 songs and the NBA table" doesn't require a specific
order.
3. If dependencies appear but are not essential, treat components as independent.
Example: "Provide New Delhi's AQI, then confirm Mailcheap API status" involves
separate tasks despite the dependent phrasing.
4. Reframe queries for grammatical clarity.
Example: "romantic movie, rnb music, with partner" becomes "Suggest a romantic movie
and Rhythm & blues music to watch with my partner."

Remember, you can NEVER contradict explicit requirements or omit critical details.
Call the Finish function once done.
```

4.2. Self-Refine

When the solution produced in the first round of retrieval and solving is deemed to have failed by the evaluator, the system enters the Reflection pipeline to determine the next steps, such as reformulation or improved retrieval. In this pipeline, the system incorporates feedback from the evaluator, which helps the LLM understand where the failure occurred and guides its decision-making process for the next iteration.

We incorporate the principle of Self-refine into our Reflection pipeline by feeding the evaluator's feedback and, when available, the provisionally generated answer into the retriever agents for further action. Our hypothesis is that reviewing the provisional answer can provide the system with a clearer understanding of why the process failed, thereby facilitating a more effective refinement process.

The intermediate answer generated during the first pass serves as a critical reference for the model to analyze and assess its performance. By generating an initial response, the LLM can pinpoint areas for improvement, such as incomplete reasoning, unclear phrasing, or incorrect facts. This intermediate step allows the model to break

Table 4.1.: Examples of Query Decomposition

Input Query	Decomposed Query
Who is the all time top scorer of the team that won the 2008 UEFA Champions League?	First, find the team that won the 2008 UEFA Champions League. Next, identify who is the all time top scorer of that team.
Who is the longest serving active head of state among countries that use Euro?	Please provide a list of the countries that use the Euro currency, then identify the current head of state for these countries, and finally determine who among them is the longest-serving active head of state.
xmas movie carols family cold	Please recommend a Christmas-themed movie and Christmas carols to enjoy with family during cold weather.
amazon smartphones under INR 20k, 4+ , 50k reviews with name, price, rating etc	Please provide the names, prices, and ratings of smartphones on Amazon priced under INR 20,000, that have a rating of 4+ stars, and have received at least 50,000 reviews.

down its own output, uncover errors that might be overlooked, and make targeted refinements. It forces the model to confront its limitations, guiding it towards a more refined and coherent final answer, much like how humans refine their work through revisions.

In cases where the solver fails to generate an answer in the first pass and simply gives up, the principle of Self-refine cannot be applied by design. In such cases, the system defaults to its original behavior and only incorporates the solver’s feedback to explain why it failed to produce an acceptable answer.

4.3. Query Reformulation Agent

Building upon the AnyTool framework, we structure the API retriever in a hierarchy of specialized agents: Meta, Category, Tool, and the newly introduced Query Reformulator agent. Each agent operates autonomously, following specific rules while making decisions about which functions to call based on function outcomes and conversation history. The Query Reformulator agent plays a pivotal role during the reflection phase, reinforcing the principle of Self-refine.

Extended dialogues often challenge LLMs in maintaining coherence and relevance due to their limited capacity for tracking long-term context, as noted by Maharana et al.[32]. In applications involving tool use and question answering (QA), sustaining accuracy across multiple turns can be especially difficult. Long conversation histories may result in earlier information being overlooked or a shift in focus, causing the system to drift from the conversation’s current direction. This phenomenon, referred to as the *Lost in the Middle*, as described by Liu et al.[45], highlights the tendency of systems to focus on initial details and neglect crucial information in the middle. In our system, during a single query resolution cycle, the conversation history may become cluttered with system messages, tool usage, and error logs, making it difficult to maintain focus. The purpose of the Query Reformulation agent is to streamline this process by summarizing previous attempts, ensuring that the conversation history remains clean and focused on critical information. To refine the query, the agent rewords it based on feedback from the evaluator, eliminating components of a multi-component query that have already been solved. This reformulation not only clarifies the remaining unresolved parts but also guides the retriever by indicating which APIs failed during the first round and how to avoid them in subsequent retrieval attempts. By focusing on unsolved parts of the query, the agent enhances the efficiency of both the retriever and the solver, avoiding redundant work.

For effective reformulation, the solver’s conversation history is refreshed by removing references to the initial query, prior thoughts, and evaluator feedback. Retaining old context may skew the results, as it is akin to giving the solver a “cheat sheet” that could influence the decision-making process. The Query Reformulator agent is instantiated by the Meta agent during the reflection phase, whenever it is deemed beneficial to rephrase the query based on interim answers and evaluator feedback. In this phase, the planner LLM is prompted with instructions to eliminate the components of the query that have already been addressed. The agent is also guided to include information about any APIs to avoid, such as those that previously failed due to issues like connection timeouts or invalid URLs. These instructions are demonstrated using few-shot

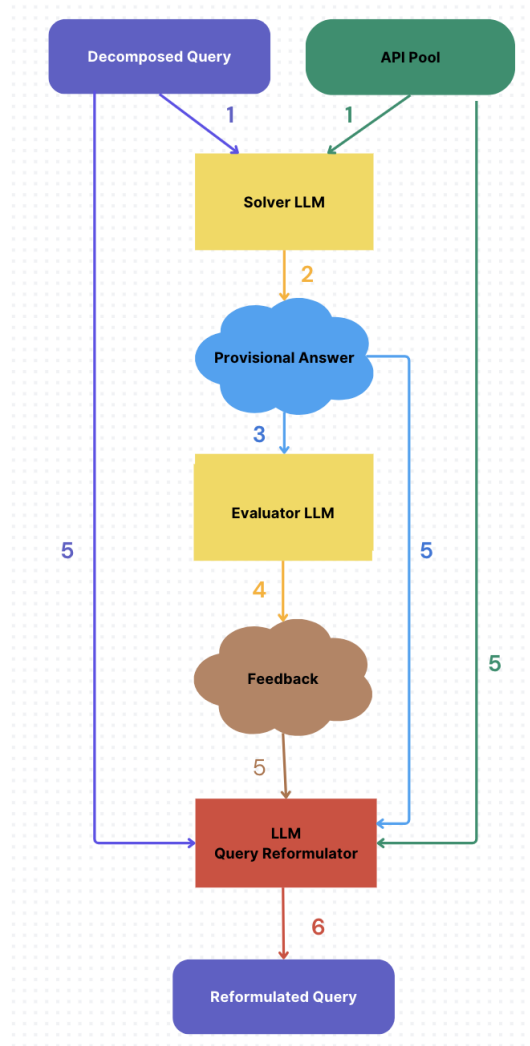


Figure 4.2.: Workflow of the Query Reformulator

prompting, which provides examples of ideal query reformulations.

To ensure quality, after reformulating the query, the LLM is prompted to assess whether the new query is a valid rewrite of the original, considering both the provisional answer and feedback from the first iteration. The reformulated query is adopted if the LLM affirms its validity. To control token usage, we limit the reformulation process to one per input query to avoid unnecessary rewrites and reflection cycles that may result in diminishing returns.

Despite the Reformulation step, the system always uses the original user input query to evaluate the correctness of answers, whether provisional or final.

4.3.1. System Prompt for Query Reformulator Agent

You are APIGPT, tasked with reformulating queries based on feedback and provisional answers from the first attempt to solve them. Your goal is to focus on unresolved components and avoid redundant effort. Ensure all unsolved components are included and incorporate feedback about why the previous attempt failed:

1. Ignore resolved components of the original query if they are independent of the unsolved parts.
2. Incorporate relevant tools or APIs used previously for unfinished components, noting which ones were unsuccessful and should be avoided. Be specific. Avoid vague terms like "several".
3. Exclude irrelevant APIs/tools from the reformulated query.
4. Retain crucial details from the original query for solving the unresolved parts.
5. If no components were adequately addressed, rewrite the query, integrating the feedback and provisional answer.
6. Do not contradict explicit requirements in the original query.
7. Whenever possible, decompose the query into logical steps and guide the solver on the order of solving.

Examples:

Original Query: What are the latest global figures for confirmed COVID-19 cases, recoveries, and fatalities?

Provisional Answer: There were 10,122 new cases of COVID across all continents in the first week of December 2024.

Reformulated Query: What are the latest global figures for recoveries and fatalities ? Avoid Covid-19 Live data APIs from the Data category.

Original Query: I need the AQI for New Delhi and to confirm Gmail is operational.

Provisional Answer: Gmail is operational.

Reformulated Query: What is the latest AQI for New Delhi and its associated health hazards?

Original Query: I am Michael Scott and need an AI to generate a resume for my profile. I am 40 years old and a Regional Manager of a paper company.

Provisional Answer: No APIs could be reached.

Reformulated Query: Generate a resume for Michael Scott, a 40 year old Regional Manager of a paper company. Avoid failed tools like Indeed and LinkedIn from the Education category.

Use the `rewrite_query` function for reformulated queries. If feedback cannot be integrated without drastic changes, call the `Finish` function.

5. Experimental Setup

This chapter provides an in-depth overview of the experimental framework, detailing the tools and APIs hosted via RapidAPI, the dataset with all queries to benchmark the system, the primary performance metric employed, and the criteria used to partition the test data.

5.1. Tool repository

RapidAPI is a prominent website that hosts a vast collection of REST APIs, facilitating the seamless integration of various functionalities into applications. REST APIs, or Representational State Transfer APIs, allow applications to request and exchanging specific data. Through a single API key, developers can register, test, and connect with APIs, easing integration. APIs in RapidAPI are organized into 49 high-level categories such as sports, finance, data etc., making it easy for developers, or as in our case, LLMs, to locate APIs relevant to their project needs. These categories contain over 10,000 tools. Each "tool" in RapidAPI is a structured unit from a source website host and contains multiple APIs, with detailed metadata for each API, including the name, HTTP methods, parameters, code snippets, and example responses. This structure allows developers to interact with APIs through standard HTTP methods like GET, POST, PUT, and DELETE to create, read, update, or delete data resources, making them versatile and easy to integrate. This comprehensive metadata helps understand and interact with APIs effectively, even in a zero-shot setting.

A common issue is unsubcription errors, where access to an API becomes unexpectedly restricted, disrupting applications that depend on it. Timeout errors can also be problematic, occurring when an API fails to respond within a set time frame, often due to server load or network instability. This delay can severely impact performance, especially in time-sensitive applications. Additionally, incomplete or poorly written documentation can hinder effective integration, forcing the system to rely on trial and error, which increases the risks of failure. To mitigate these risks, a stringent filtering process by the Qin et al. [36] reduced the initial set of 53,190 APIs across 10,853 tools to a relatively more dependable subset of 16,464 APIs across 3,451 tools, ensuring high-quality APIs are retained for robust application development.

Despite this filtration process, conducted in 2023, many APIs have since become unreliable because of similar issues, and lack of maintenance by the tool providers at the source. However, these non-functioning tools are still retained in the API repository for the system to retrieve and attempt to use for the purpose of testing its flexibility with bad API retrieval, and to maintain consistency with other benchmarks.

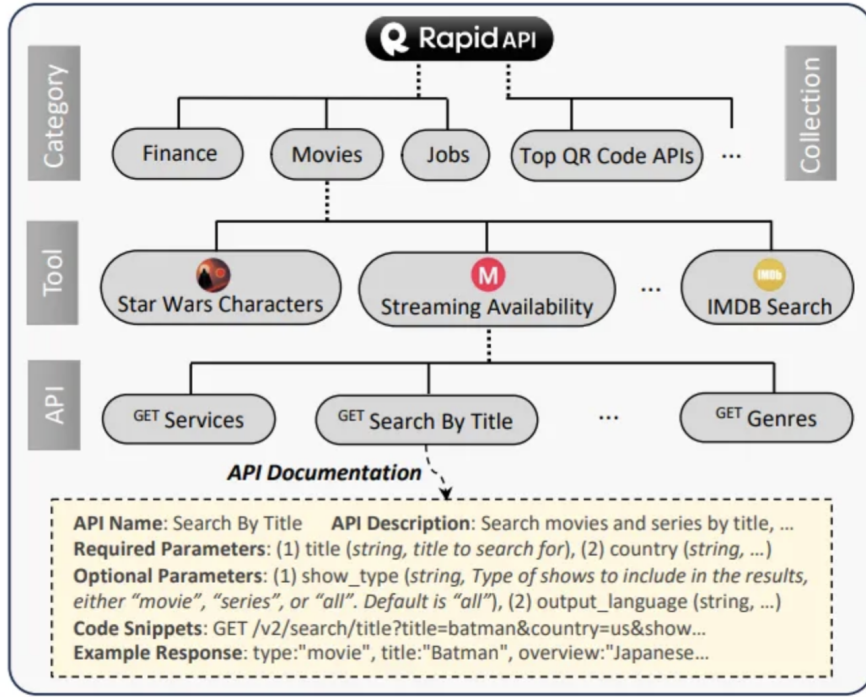


Figure 5.1.: Hierarchy of RapidAPI [36]

In order to obtain results without having to manually subscribe to individual APIs, the maintainers of ToolLLM provide a personal Toolbench key to enable requests to be made to a single unified server. External API calls to tools on RapidAPI are made by sending POST requests with a structured JSON payload to a designated URL. Each request includes fields specifying the tool's category, name, API endpoint, input parameters, response processing instruction, and the authorization Toolbench key for access. Based on the documentation of tools found on RapidAPI, the parameter fields are defined at the time of converting the shortlisted APIs into OpenAI function versions, and automatically filled by the GPT model at the time of calling the function. The exact design of this system and procedure is left out of this thesis as it is beyond its scope.

5.2. Datasets

5.2.1. Organization

We rely on subsets of 2 datasets to benchmark the performance of our system:

- ToolBench, developed alongside ToolLLM by Qin et al. [36]
- AnyToolBench, developed by Du et al. [42].

ToolBench is organized into 3 partitions:

- G1 - Comprising queries that require only a single tool’s APIs to be solved.
- G2 - Comprising queries that require multiple tools across a single category to be solved.
- G3 - Comprising queries that require multiple tools across more than one category to be solved.

5.2.2. Generation

Each query in Toolbench was generated by Qin et al.[36] by sampling a small number of APIs and their documentation, and feeding these to ChatGPT with instruction prompts to understand the functionality of the sampled APIs, and generate queries that involve the sampled APIs. These generated queries were split into train and test sets to train the API Retriever, but since our system does not depend on a trained API Retriever, it is possible to benchmark it using the superset of queries.

The prompt provided to ChatGPT for generating queries contained a high-level explanation of the instruction generation task (System prompt), detailed API documentation to help ChatGPT grasp the functions and relationships between APIs, and finally, three human-created instruction samples as in-context examples to serve as ideal models for generation.

Similarly, Du et al.[42] generated a different benchmark, AnyToolBench, by equipping GPT-4 with various functions to navigate the API repository, including functions to filter tools by category, or obtain tool descriptions, list APIs within a tool, and view specific API details. GPT-4 then selected and added APIs to a candidate pool, and finally generated parameters for these APIs to formulate queries based on actual API responses. Additionally, GPT-4 was guided to produce a solution for each query, to minimize hallucinations by grounding the queries in API responses, with

the solutions validated to ensure quality. In addition to creating AnyToolBench, they highlight queries in ToolBench which were deemed by the LLM to be *Unsolvable* for being incomplete in details. For example:

- "My company needs to monitor the progress of multiple packages. Can you help us count the number of steps in the history for each package? This will allow us to optimize our resources and network consumption."
- "Our firm is launching a new product and we want to create a promotional video. Can you recommend videos related to similar products that have been successful on Youtube? Please provide me with the video details, including the title, author, number of views, and average rating for each video. Additionally, I would like to see the available subtitles for these videos."
- "I'm a teacher planning a field trip for my students and need some information. Can you provide me with a list of educational attractions in the area, the cost of admission for students, and recommendations for nearby restaurants? Additionally, we would like to see some pictures of previous field trips to inspire us."

These queries were excluded from ToolBench to avoid the issue.

5.2.3. Sampled subset of queries

Due to OpenAI usage limit constraints, we limit our analysis to a random sample of 80 queries in the dataset. These queries are retrieved from AnyToolBench and the Toolbench G1, G2, G3 list of queries above. To ensure a diverse set of queries, we sample indices to approximate a balanced distribution of queries based on the number of tools involved in the ground-truth list of APIs associated with the query - 1, 2, or 3+ more tools. This approach resulted in 28 single-tool queries, 46 dual-tool queries, and 30 multi-tool queries, providing a variety of tool use scenarios for benchmarking.

The queries range from simplistic queries requiring only 1 category of APIs such as *"Can you check if the domain 'getstartedparty.com' is a disposable or valid email domain?"* to complex queries that require 3 categories such as *"Can you provide an update on the current COVID-19 global statistics, the top song from the U.S. Afrobeats Billboard chart for the most recent week available, and the latest weather conditions including temperature and air quality index in Lagos, Nigeria?"*

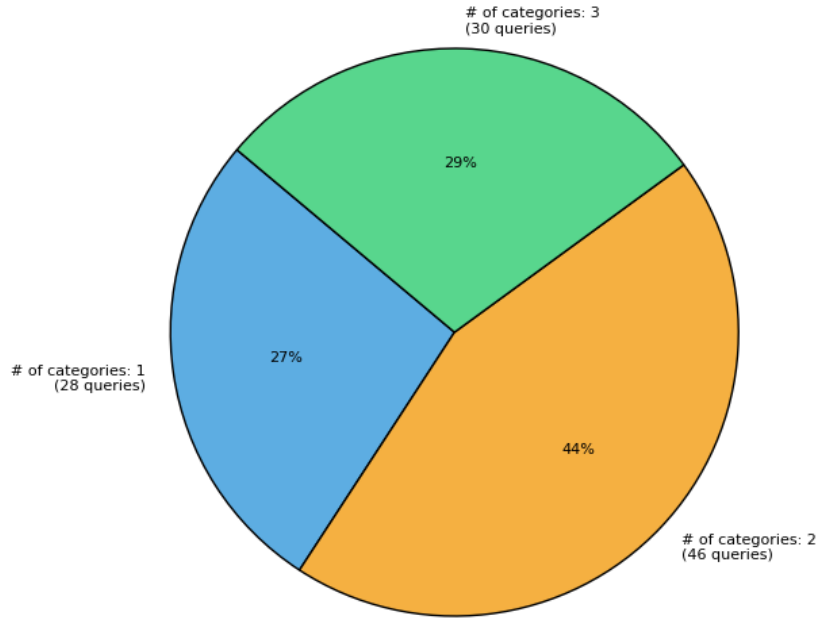


Figure 5.2.: Queries - Grouped by number of Categories in ground truth list of APIs

Some of these queries require multiple tools, even if only from a single category, such as *"I need to know the sunrise and sunset times in London on November 7, 2023. Additionally, I require the weather forecast for London on that date, with a focus on temperature and precipitation, as well as any air quality information available. Finally, please provide an interesting piece of trivia related to geography."* which requires 2 tools from the category Weather - one for the weather forecast and another for the times for sunrise and sunset, and another from the category Geography.

The portion of the dataset we use to benchmark performance comprises of multiple queries associated with categories like Media, Health, Finance, Sports, Weather, Mapping, among others. 47 out of the 49 total categories in RapidAPI are covered in the list of ground truth APIs associated with our sampled dataset. This provides a glimpse of the range of topics that the system is equipped to tackle and navigate to provide a satisfactory resolution.

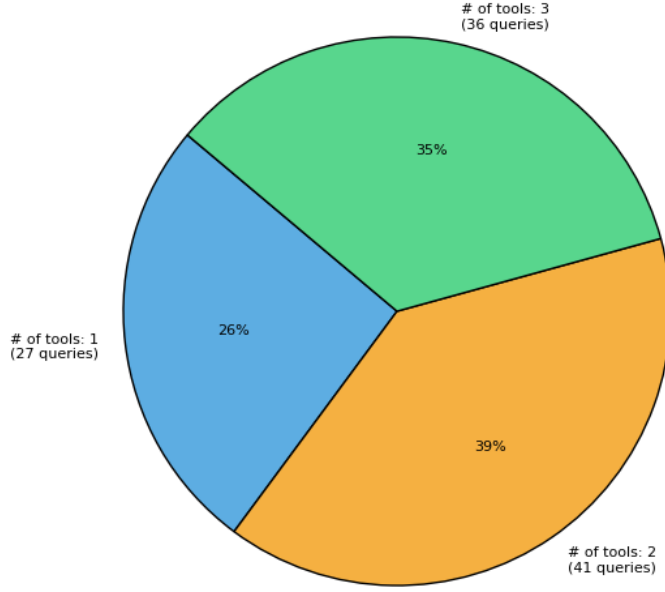


Figure 5.3.: Queries - Grouped by number of Tools in ground truth list of APIs

5.3. Metrics

We measure the quality of decomposition and retrieval using the Solvable @ 1. In order to do so, first, the retriever process the query to shortlist a list of suitable APIs, at which point, the LLM is provided the decomposed query and the shortlisted APIs with documentation, and asked to judge if the query is "Solvable" with those APIs. The query is deemed "Unsolvable" if it requires information that the shortlisted APIs do not provide, or if the query provides incomplete information such as invalid details. Alternately, the query is deemed "Unsure" if the query is unethical, illegal, or the system is unable to categorize it into Solvable or Unsolvable.

The Solvable @ 1 is calculated as the fraction of queries that are deemed Solvable. In a modification from AnyTool's calculation, we exclude the cases of queries being deemed solvable during the reflection process through a new API search. We only consider those queries solved which are deemed Solvable **before the solver is called for the first time**.

$$\text{Solvable @ 1} = \frac{\text{Solvable (after first retrieval attempt)}}{\text{Solvable} + \text{Unsolvable} + \text{Unsure}} \quad (5.1)$$

Once the APIs are shortlisted, the query and API pool are sent to the solver to generate a solution. We use Pass Rate, introduced by Qin et al.[36] for benchmarking the performance of their trained retriever and model.

The query is deemed "*Passed*" if the generated solution is judged to be valid. The Pass Rate is calculated as the sum of all queries deemed *Passed* and all queries deemed *Unsolvable* as a fraction of the total number of input queries.

$$\text{Pass Rate (ToolLLM)} = \frac{\text{Unsolvable} + \text{Passed}}{\text{Unsolvable} + \text{Passed} + \text{Failed}} \quad (5.2)$$

The thought behind this calculation is to not penalize the system for queries that are incomplete and impossible for even ideal retrievers and solvers to solve without contradicting explicit instructions or making questionable assumptions.

However, Du et al. [42] observed that it artificially inflated the Pass Rate by allowing the retriever to underperform by not retrieving appropriate tools for any query but still providing a high Pass Rate. They recommend a workaround where Pass Rate is calculated as all queries deemed *Passed*, as a fraction of all the queries attempted.

$$\text{Modified Pass Rate (AnyTool)} = \frac{\text{Passed}}{\text{Total Queries Attempted}} \quad (5.3)$$

For our experiments, we use the latter definition to calculate Pass Rate.

5.4. Model

For planning the retrieval process, solving the problem with retrieved APIs, and evaluation of generated answers, we work with OpenAI's GPT-4 text generation models. Specifically, we choose GPT-4o [46], which is comparable to the reasoning prowess of GPT-4 Turbo. It offers a balance between intelligence and price, at half the cost of the Turbo model, and is 2x faster.

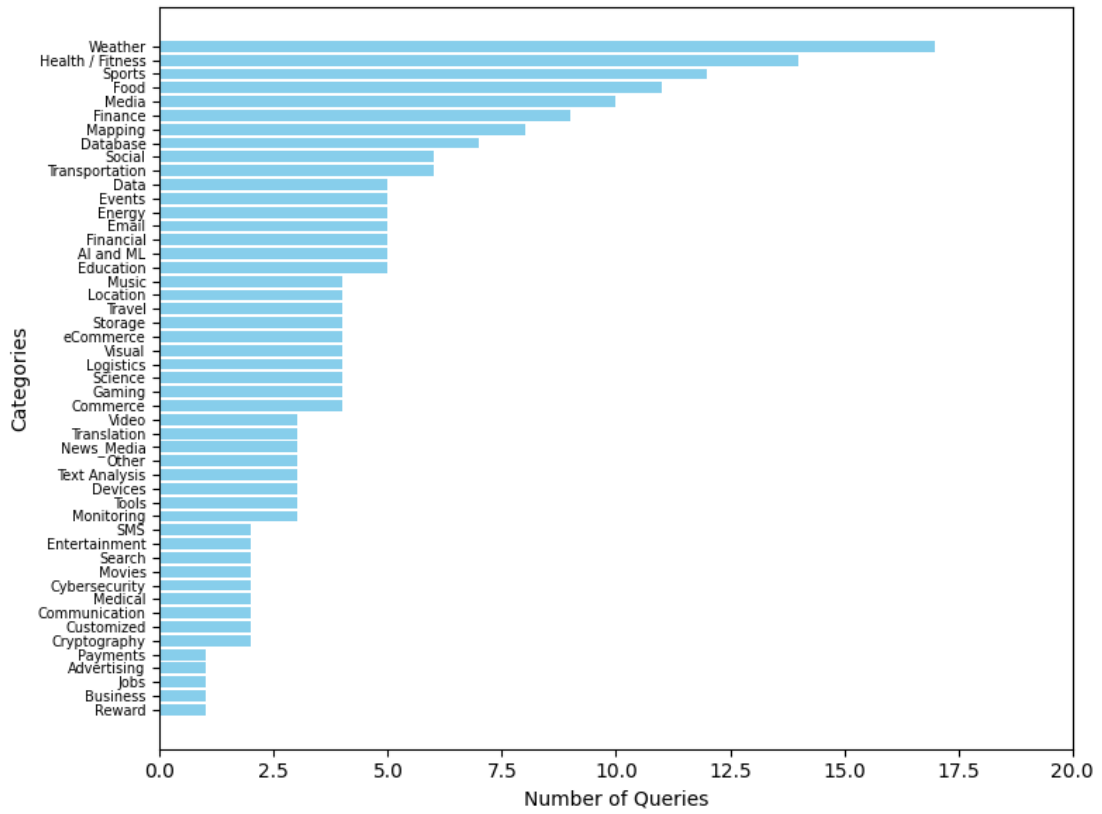


Figure 5.4.: Frequency of categories in dataset

6. Results and Errors

In this section, we present and interpret the results of our experiments listed under the settings listed in the previous chapter. Results are compared across queries split by complexity to measure the difference between the baseline AnyTool and our system with the Decomposer, Self-refine and Query Reformulator Agent. Additionally, we determine if the difference is statistically significant and explore the kind of errors that are observed.

6.1. Overview across pooled queries

First, we compare the overall Solvable @ 1 and Pass Rate across all 104 queries on the baseline and our system.

Metric	AnyTool (Baseline)	Proposed system
Solvable @ 1	74%	78%
Pass Rate	62%	72%

Table 6.1.: High-level comparison with Baseline

From the surface, we see that our proposed system improves upon the performance of our baseline, more prominently in the Pass Rate (Solver side of the framework) than the Solvable @ 1 (Retriever side).

6.2. Queries grouped by number of required tools

Next, we inspect the performance of the system across the queries grouped by the number of tools that are part of the solution’s ground truth list of APIs - that were actually used to generate the query in the first place. This acts as a proxy for query complexity, where more tools suggest harder problems to solve in entirety.

6.2.1. Solvability

We observe that across frameworks, the retrieval quality declines as queries become more complex and require more tools. However, for our limited dataset, our proposed system with 3 additional components is seen to select tools and APIs at a better rate than AnyTool, and the performance diverges more as the query complexity rises.

Tools Req'd.	AnyTool (Baseline)	Proposed system
1 Tool	89%	89%
2 Tools	73%	78%
3+ Tools	63%	69%

Table 6.2.: Solvable @ 1 across queries grouped by complexity

Solvable in Our System but not Baseline

- "I am hosting an event for digital nomads in Lisbon and want to serve a simple cocktail for 6-8 people. Can you give me the exact recipe for an 'Aperol Spritz' from The Cocktail DB? Also, for a discussion on mobile work tools, what are the full specifications, including pictures, of the 'Acer Chromebook Tab 10' which from the GSMarena database?"
- "I drove a petrol-fueled car using 50 liters of fuel, and as an Aries, I'm curious about my compatibility with a Leo. Can you tell me what my carbon footprint from this trip is and provide an overview of how well my personality meshes with someone born under the sign of Leo, in terms of our fiery dispositions and mutual respect when taking charge?"

*The queries that are found to be Solvable @ 1 in our proposed system but not the AnyTool baseline include the above. Some of these are observed to contain **very long sentences that contain some irrelevant details** such as the bit about nomads in Lisbon, that the retriever feels compelled to explore anyway, which takes away focus from the right categories and tools to search.*

Solvable in Baseline but not Our System

- "Can you show me smartphone options available on Amazon with a price range of Rs. 7,000 to Rs. 15,000 and provide any cybersecurity insights on a suspicious IP address 103.242.175.229? Please include the specifications and details of the smartphones as well as any cybersecurity concerns associated with the IP address."
- "I'm planning a birthday celebration and would like to find the contact for a restaurant called 'La Alameda' in San Francisco, California, to book a table. Additionally, I'm considering personalizing a gift for the birthday person with the baby name 'Elliot', and I'd like to confirm if it is a popular baby name for boys recently."
- "What are the total and new COVID-19 cases reported in the United States on the 12th of March, 2020, and could you also provide a human-readable address near the coordinates 37.7749, -122.4194? Additionally, please confirm if Google Translate supports translating from English to Azerbaijani."

*In the alternate case, the queries for which the baseline retriever was successful but our proposed retriever with Query Decomposition was unsuccessful include those like the above where the **correct category was identified** but out of multiple possible tools, an **unsuitable tool** was shortlisted to choose APIs from. Additionally, these are queries requiring multiple tools, hence have many points of failure.*

6.2.2. Pass Rate

We also observe that the ability of the LLM to use the selected APIs to make external API calls to solve the query also declines as the number of required tools increases. However, our proposed system performs better than AnyTool more as the query complexity rises - with the same caveat of the dataset being relatively small.

Tools Reqd.	AnyTool (Baseline)	Proposed system
1 Tool	86%	89%
2 Tools	63%	71%
3+ Tools	40%	60%

Table 6.3.: Pass Rate across queries grouped by complexity

Passed in Our System but not Baseline

- "For a business presentation on global trends in music and sports performance analysis, could you provide the top streaming songs on Spotify for the most recent available global chart data, along with the corresponding 'hello world' placeholder text that will be used for introducing programmatic greetings, and the win-loss records for NFL teams from the 2022 season to illustrate the competitive landscape?"
- "I'm interested in buying the Apple iPad with the ASIN B08J65DST5, the 10.2-inch 8th Generation model, from the US Amazon store. Could you tell me the current price in Euros, and also list the countries in Europe where the Euro is accepted as currency?"
- "What is the schedule for the next NFL match featuring the Tampa Bay Buccaneers versus the Kansas City Chiefs and what is the latest version of the brand recognition software currently available?"

*Some of the queries that are found to have Passed in our proposed system but not the AnyTool baseline include the above. These queries are often those with multiple, unrelated components, **some of which are solved in the first pass with the baseline's solver**, but wasting resources on addressing them even during reflection can **draw away focus from missing information**.*

Passed in Baseline but not Our System

- "Can you check the numerological health status, evaluate the love compatibility for the names 'Alice' and 'Bob' with birth dates '1990-05-15' and '1988-11-23' respectively, and provide the detailed weather forecast map tile for an outdoor dating spot at coordinates (X:10, Y:20, Zoom Level: 5) with metric units, as we plan our day together?"
- "I'm interested in the current prices of a T-fal Initiatives Nonstick Cookware Set and a Maxracy 2 Set Peeler on Amazon US. Can you provide me with the most recent prices along with the product names, whether they have Prime eligibility, and the product images for these specific ASINs: B005YQZ1KE and B074R8RQQ2?"
- "Can you give me an analysis of the latest news sentiment regarding Apple (AAPL) stock, the leading hashtags currently trending in the United States on Twitter, and whether there are any upcoming public holidays in the U.S. that might affect the stock market trading hours or investor behavior?"

*On the other hand, the queries that could be solved by the baseline but not by our proposed system are typically associated with selection of tools or APIs that are **otherwise suitable and functional**, but happen to not contain the exact data required. For this reason, they are not classified as retrieval errors because the retriever cannot be expected to account for these given the existing documentation. Another reason for failure is **incorrect parameter formatting** when making external API calls. This kind of error is common to both our proposed system as well as the baseline.*

6.3. Ablation Study

In this section, we measure the difference made by the absence of each component of our system - Decomposition, Self-refine, or the Query Reformulation Agent.

These results show that the Query Reformulator Agent is the biggest difference making component, while Self-refine makes the smallest difference.

Tools Req'd.	Without Decomposer	Without Self-refine	Without Reform. Agent
1 Tool	89%	89%	86%
2 Tools	63%	68%	61%
3+ Tools	54%	57%	49%

Table 6.4.: Pass Rate on removing one key component from our system

Query Decomposer - Use Case 1

Original query: "comedy flm + lively rnb music to enjoy with fam holiday"

Decomposed query: "Please recommend a comedy film and 'Rhythm and blues' music to enjoy with family on holiday."

Final answer: "***Movie Suggestion**": Title: 'Clean' (2022) - A comedy movie available for streaming on AMC+ Amazon Channel or IFC Films Unlimited Apple TV Channel. You can also rent it on platforms like Apple TV and Amazon Video. ****Music Suggestion****: Playlist: 'Family party music', a lively and good party mix: lively and chill. Available on Spotify <https://open.spotify.com/playlist/4RKIZXWxHWA1bBsMxDqRQ7>. Enjoy your holiday with these family-friendly choices!"

*This is an example of a query that works in our proposed system but not without the Query Decomposer. The query is written in **shorthand English** and benefits from being reframed which tells the retriever exactly what information is expected from the tools.*

Query Decomposer - Use Case 2

Original query: "If a 35-year-old male living in Punta del Este, Uruguay, with a height of 175 cm and a weight of 70 kg, who has a normal BMI of 22.9, wants to evaluate the quality of the air for planning his physical activities, could you provide the current air quality index, the primary pollutant identified, and both the recent historical and expected air quality trends over the next days based on his specific location of latitude -34.9485 and longitude -54.9341?"

Decomposed query: "Please provide the current air quality index for Punta del Este, Uruguay, located at latitude -34.9485 and longitude -54.9341, along with the primary pollutant identified. Additionally, include the recent historical air quality trends and the expected air quality trends for the upcoming days for this specific location, so that a 35-year-old male can evaluate the air quality for planning his physical activities."

Final answer: "The current Air Quality Index (AQI) for Punta del Este, Uruguay is 26, indicating good air quality with Ozone (O3) as the primary pollutant. Historical data from the past 24 hours shows consistent AQI levels between 26 and 30, largely influenced by O3 levels. The forecast for the next few days predicts the AQI to remain in the good range (25-26), with O3 continuing as the predominant pollutant, ensuring a continuance of favorable air quality conditions."

*This is an example of a query that **contains unnecessary details** about BMI and age, which have no practical relevance to the essence of the question. The decomposition **eliminates unnecessary focus** on these details and tools related to BMI from being instantiated.*

Query Reformulator Agent - Use Case 1

Original query: "Can you provide detailed information about "The Incredible Hulk" movie that was released in 2008, including its plot, genres, and how it's evaluated by audiences, and also tell me the current timezone for Los Angeles, USA?"

Provisional answer: "The Incredible Hulk" is a movie released in 2008. It falls under the genres of Action, Adventure, and Sci-Fi. The movie has an aggregate rating of 6.6 based on 530,282 votes. The plot revolves around the scientist Bruce Banner who, after being exposed to gamma radiation, transforms into the Hulk whenever his pulse rate rises. As he searches for a cure, those who dream of exploiting his powers try to capture him. Here's an image from the movie featuring Edward Norton: m.media-amazon.com/images/M/MV5BMTUyNzk3MjA1OF5BMl5BanBnXkFtZTcwMTE1Njg2MQ@@._V1_.jpg

Reformulated query: "Can you tell me the current timezone for Los Angeles, USA?"

Final answer:

Answer 1) - "The Incredible Hulk" is a movie released in 2008. It falls under the genres of Action, Adventure, and Sci-Fi. The movie has an aggregate rating of 6.6 based on 530,282 votes. The plot revolves around the scientist Bruce Banner who, after being exposed to gamma radiation, transforms into the Hulk whenever his pulse rate rises. As he searches for a cure, those who dream of exploiting his powers try to capture him. Here's an image from the movie featuring Edward Norton: m.media-amazon.com/images/M/MV5BMTUyNzk3MjA1OF5BMl5BanBnXkFtZTcwMTE1Njg2MQ@@._V1_.jpg

Answer 2) - The current timezone for Los Angeles, USA is America/-Los Angeles, Pacific Standard Time, GMT-8.

*This is an example of a query where the extensive details required in one part of the query means another part is left unaddressed - a phenomenon we call **Tunnel Vision** - in the first pass. The reformulation clears the solved portion of the query to refine focus in the second pass.*

Query Reformulator Agent - Use Case 2

Original query: "Can you give me the specifications of a Boeing 777 200ER, such as how many passengers it can hold, its cost in USD, fuel capacity, max takeoff and landing weights, empty weight, overall length, wingspan, cruising speed, and flight ceiling? Additionally, what are the exact coordinates of where this plane is manufactured, the Boeing Everett Factory?"

Provisional answer: None

Reformulated query: "Can you give me the specifications of a Boeing 777-200ER, which include passenger capacity, cost in USD, fuel capacity, max takeoff and landing weights, empty weight, overall length, wingspan, cruising speed, and flight ceiling? Additionally, I need to know the exact coordinates of where the Boeing 777-200ER is manufactured, specifically the Boeing Everett Factory. Previously unsuccessful APIs involved were from Aircraft data simple collection and Flightera Flight Data in the Transportation and Travel categories."

Final answer: The Boeing 777-200ER can accommodate up to 314 to 396 passengers. It is priced around 261.5 million USD. The fuel capacity of this aircraft is approximately 31,000 US gallons. In terms of weight, the maximum takeoff weight is about 656,000 pounds, and the maximum landing weight is around 492,000 pounds. Its empty weight stands at approximately 307,000 pounds. The Boeing 777-200ER has an overall length of 209 feet and a wingspan of 199 feet 11 inches. It cruises at a speed of around 560 miles per hour (0.84 Mach) and has a flight ceiling of 43,100 feet. The Boeing Everett Factory, where this plane is manufactured, is located at the coordinates 47.9245° N, 122.2989° W.

*In this query, the solver is unable to produce any answer in the first pass because of **faulty APIs shortlisted which produce no provisional results**. As a result, the reformulation does not eliminate any details from the query but flags off the unsuccessful APIs to explore beyond them.*

6.4. Statistical Significance

To evaluate the significance of the observed improvement in success rates between the baseline method and our proposed system, we perform a two-proportion z-test [47]. This test is well-suited for comparing proportions from two independent samples

under specific conditions. The results of our experiment are binary, i.e either Solvable or Not Solvable, or Passed or Failed. Each individual query is independent, satisfying another assumption for validity of this test.

The hypotheses for the experiment are defined as follows:

$$H_0 : p_{\text{modified}} \leq p_{\text{baseline}}$$

$$H_a : p_{\text{modified}} > p_{\text{baseline}}$$

Where p_{baseline} represents the success rate of the baseline method, and p_{modified} represents the success rate of our proposed system with the 3 additional components. The statistical significance level was set at $\alpha = 0.05$.

This is a one-tailed test because the goal of the experiment is to evaluate whether the modified method has a higher success rate than the baseline method. The alternative hypothesis (H_a) is directional, focusing exclusively on improvements in the success rate ($p_{\text{modified}} > p_{\text{baseline}}$). Testing for a decrease in success rate is not relevant in this context, as the modification is intended to yield an improvement.

6.4.1. Overall Performance

The analysis of system performance revealed that both *Solvable @ 1* and *Pass Rate* were nominally higher in the proposed system compared to the baseline, revealing potential improvement in its ability to solve complex queries. However, the one-tailed two-proportion z-test indicated that these improvements were not statistically significant at the 5% significance level when the overall performance across all 104 queries is compared. This finding suggests that despite the improved numbers on our dataset with the new components, the overall performance gains may not be sufficient to reject the null hypothesis that there is no significant improvement in our proposed system. Further research with larger datasets may help confirm the system’s impact more definitively.

Metric	AnyTool (Baseline)	Proposed system	p-value
Solvable @ 1	74%	78%	25.8%
Pass Rate	62%	72%	5.3%

Table 6.5.: Neither Solvable @ 1 nor Pass Rate are statistically significantly better in our proposed system at a 5% level.

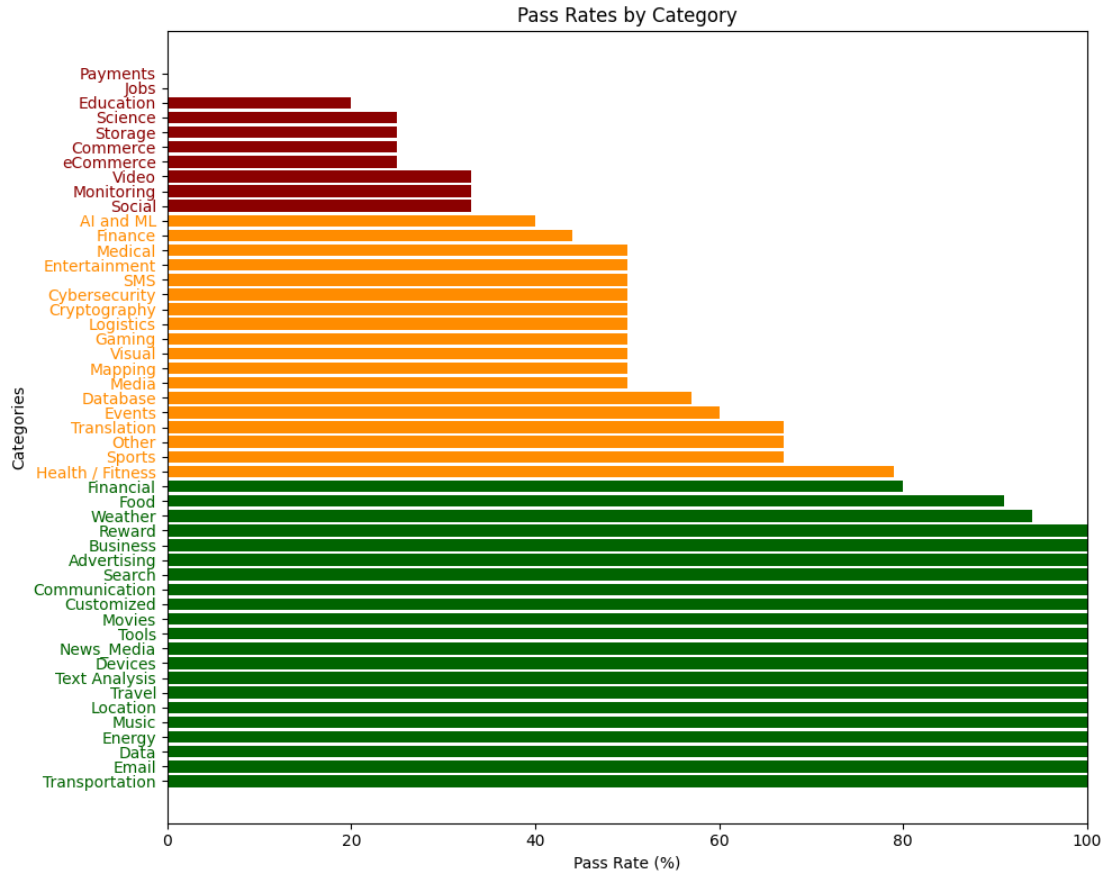


Figure 6.1.: Pass Rates across query categories in our proposed system

6.4.2. Split by number of tools

A deeper analysis of the results for queries requiring **3+ tools** revealed a statistically significant **improvement in Pass Rate** at the 5% significance level. This enables us to **reject the Null Hypothesis**, that in the case of queries requiring 3 or more tools, the Pass Rate in the baseline is greater than the Pass Rate of our proposed system. This confirms the system's enhanced ability to solve complex queries involving multiple tools.

Tools Req'd.	AnyTool (Baseline)	Proposed system	p-value
1 Tool	89%	89%	50.0%
2 Tools	73%	78%	30.4%
3+ Tools	63%	69%	30.7%

Table 6.6.: The Solvable @ 1 Rate is not statistically significantly better in our proposed system at a 5% level for any kind of query.

Tools Req'd.	AnyTool (Baseline)	Proposed system	p-value
1 Tool	86%	89%	34.3%
2 Tools	63%	71%	24.0%
3+ Tools	40%	60%	4.7%

Table 6.7.: The Pass Rate is statistically significantly better in our proposed system at a 5% level for queries that require 3 or more tools.

6.5. Error Analysis

While the experiment results demonstrate the comparative performance of the baseline and our additional components, understanding the nature of errors is equally critical. This section examines the types of errors encountered during the experiments. Such an analysis provides insights into existing issues and potential limitations of the system, guiding future development and refinement efforts.

6.5.1. Error types

Errors are categorized into 2 categories based on which part of the RAG workflow they occur in:

- **Retriever Errors:** Failures arising from the tool selection system returning insufficient tools for solving the query.
- **Solver Errors:** Issues caused by the solver failing to utilize retrieved tools effectively, resulting in incorrect or incomplete answers.

6.5.2. Retriever Errors

The following types of errors were identified on the retriever side:

- The Main agent selects an entirely incorrect category, diverging from the intended context of the query. For example, Text sentiment analysis tools when the query asks for Stock sentiment.
- The query is structured in a manner that implicitly requires the retrieval of a specific tool; however, the Category Agent selects an alternative tool that does not cater to explicit requirements. - For example, a query that asks for Spotify rank data calling a Youtube or Billboard API.
- The correct tool is retrieved but the Tool Agent selects an incorrect API from the list within the tool.
- APIs are shortlisted but the retriever incorrectly judges them to be sufficient and sends the APIs to the solver, causing issues down the line that originate with the early termination of retrieval.
- API or Tool does not have the requested data despite nominally being appropriate.
- Tools that are no longer maintained by developers and are out of service (This issue is unfixable under the scope of this thesis).
- Poorly structured or incomplete documentation for otherwise functioning Tools or APIs, which limit the retriever agents from accurately judging their suitability (This issue is unfixable under the scope of this thesis).

6.5.3. Solver Errors

The following types of errors were identified on the solver side:

- Incorrect parameter formatting in the JSON payload sent alongside the HTTP POST request to the server to use the API. For example, a date argument provided in the YYYY-MM-DD format when DDMMYYYY is expected.
- Faulty query reformulation by the Reformulation agent, where essential details were omitted, resulting in an incomplete revised query. For example, leaving out a specific University name present in the original query about generating a professional resume.
- Narrow focus on solving only one part of the query (**Tunnel Vision**), resulting in getting stuck in multiple futile attempts at resolution and the neglect of other equally important aspects of the query.

- The system failed to retrieve critical information due to limitations in the response message size, exceeding the context window capacity.
- Unusual errors with HTTP POST output despite the API functioning correctly on RapidAPI directly.
- The model erroneously considered an API failure message as an acceptable completion of burden, marking the task as complete despite the lack of a valid solution.
- Incorrectly judging a complete answer containing inaccuracies or insufficient information to be correct.

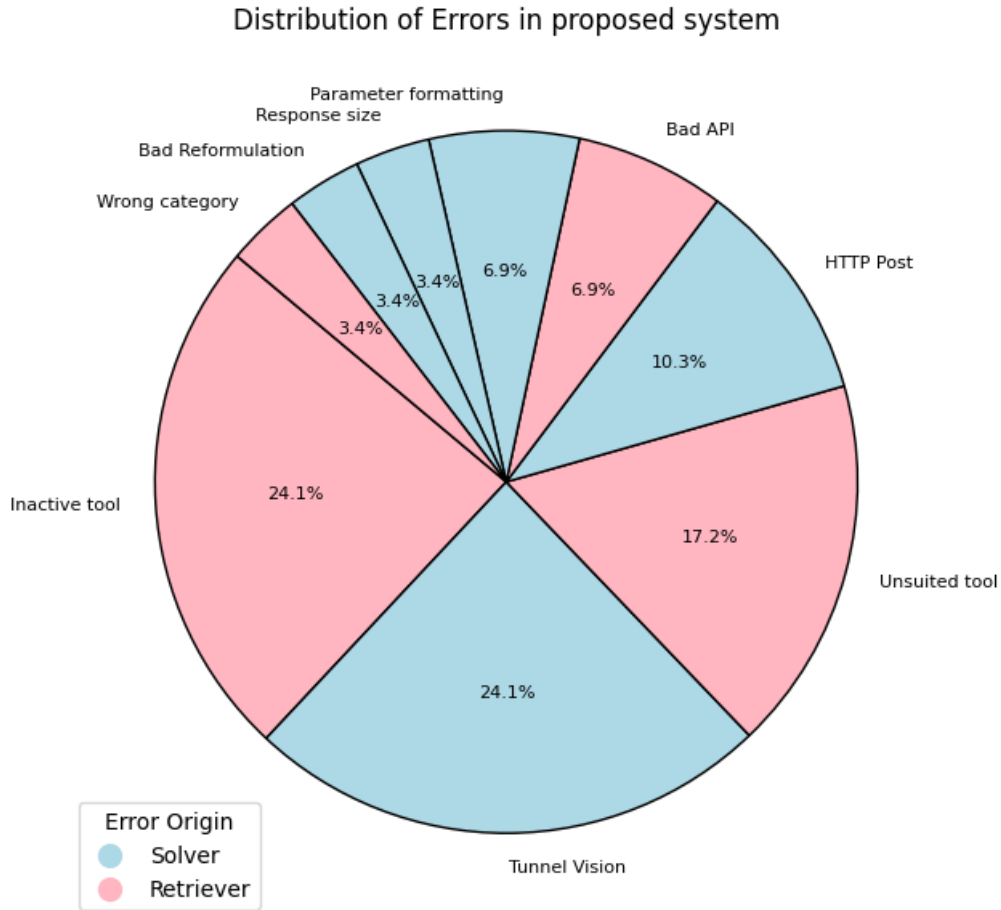


Figure 6.2.: Distribution of errors in proposed system

Break-up of Unsolved queries by answer completeness

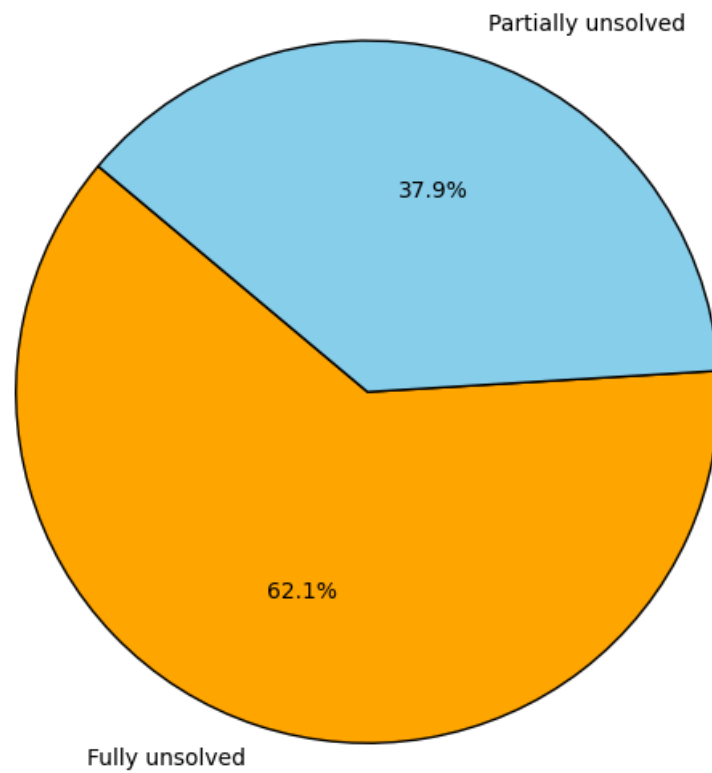


Figure 6.3.: Types of answers in unsolved queries

7. Conclusion and Future Research

In this final chapter, we aim to summarize the findings of the thesis with conclusions to the original Research Questions investigated in the thesis, and highlight potential concerns with the system that future research may aim to address.

7.1. Conclusions

Here we examine the importance of each of the three components in our proposed system.

7.1.1. Influence of Query Decomposition:

The analysis indicates that Query Decomposition improves the system’s ability to solve queries, particularly those requiring the use of 3 or more tools. The statistically significant improvement in the Pass Rate at the 5% level for such queries confirms the system’s enhanced ability to handle multi-tool tasks from end-to-end. The divergence in performance between the proposed system and the baseline as query complexity rises underscores the role of Query Decomposition in solving queries by ignoring redundant details and organizing the query into a sequence of sub-queries where possible—something not available in the baseline system.

Through ablation studies, we observe that the additional presence of the Query Decomposer improves the Pass Rate for queries requiring **more than 1 tool** by 7%–8% compared to a system containing only the Self-refine component and the Query Reformulator Agent.

7.1.2. Influence of Self-refine:

The ablation results suggest that Self-refine contributes minimally to the overall performance of the system. Its impact appears limited when considering the presence of the Query Decomposer and the Query Reformulator Agent in the system. While Self-refine does not worsen performance compared to the baseline, outsourcing the evaluation of provisional answers to a different model appears to be sufficient.

7.1.3. Influence of Query Reformulation with a fresh conversation history:

The analysis indicates that Query Reformulation improves the system’s ability to answer complex queries. The statistically significant improvement in the Pass Rate for queries requiring 3 or more tools to solve at the 5% level highlights its effectiveness. The divergence in performance between the proposed system and the baseline as query complexity rises confirms the role of Query Reformulation by removing the originally solved components from the query during reflection and explicitly calling for searching tools beyond unsuccessful APIs — features not available in the baseline system.

Ablation studies show that the additional presence of the Query Reformulator improves the Pass Rate for queries requiring **more than 1 tool** by an extra **10%–11%** compared to a system containing only the Self-refine principle and the Query Decomposer.

7.2. Future Research

In this section, we flag some potential concerns with the proposed system that future research may aim to address.

7.2.1. Token consumption

The system’s reliance on extensive token usage during decomposition, self-refine, and reformulation affect the feasibility of using it for wide practical usage. Each stage of the pipeline requires the model to generate and interpret lengthy responses, which increases the total token count. This poses challenges for systems with strict computational costs. Reducing token usage without sacrificing performance remains an area for optimization.

7.2.2. Answer generation time

The time required to generate answers is another significant concern, particularly as the system introduces processing layers, such as query decomposition and iterative feedback refinement in addition to the steps of hierarchical API retrieval. These steps, while improving accuracy and problem-solving capabilities, also increase latency, making the system less suitable for time-sensitive applications. For complex queries involving multiple tools or iterations, the response generation time can grow substantially. This delay may negatively impact user experience, especially in scenarios where rapid responses are critical - such as real-time chatbots or QA systems. Balancing the trade-off between

7. Conclusion and Future Research

enhanced performance and processing speed is essential to improving the system's practicality for real-world use cases.

List of Figures

2.1. LLMs can use external tools to enhance their capabilities [11]	6
2.2. Workflow of an autonomous agent [13]	8
2.3. Generic workflow of Query Rewriting [21]	10
2.4. Our system goes beyond naive RAGs for the rewrite to aid retrieval [22]	11
3.1. LLaTrieval uses the LLM to judge when retrieval is complete [23]	13
3.2. RePlug composes an answer as a weighted average of each individual retrieved document [25] [22]	13
3.3. Self-refine sees the LLM review its own interim output and uses that feedback to generate a new answer	14
3.4. Iter-Retgen "rewrites" the query to guide retrieval with a blind guess.[28] [22]	15
3.5. MARG [40] uses a multi-agent architecture where agents have different roles to combine for the task of reviewing a scientific paper	19
3.6. Workflow of AnyTool with a hierarchical API retriever to sift through 16,000+ APIs [42]	22
4.1. Workflow of the Query Decomposer	26
4.2. Workflow of the Query Reformulator	30
5.1. Hierarchy of RapidAPI [36]	33
5.2. Queries - Grouped by number of Categories in ground truth list of APIs	36
5.3. Queries - Grouped by number of Tools in ground truth list of APIs . . .	37
5.4. Frequency of categories in dataset	39
6.1. Pass Rates across query categories in our proposed system	50
6.2. Distribution of errors in proposed system	53
6.3. Types of answers in unsolved queries	54

List of Tables

4.1. Demonstrations of Query Decomposition	28
6.1. Overview Results	40
6.2. Query complexity - Solvability	41
6.3. Query complexity - Pass Rate	42
6.4. Ablation - Pass Rate	45
6.5. Statistical Significance - Overall	49
6.6. Statistical Significance - Solvable @ 1 by Tools required	51
6.7. Statistical Significance - Pass Rate by Tools required	51

A. Additional Prompts

A.1. System Prompt for Rewrite Validity

You will be provided a query, a provisional attempt at answering it, and a reformulation of the original query for a second attempt at solving it. Your task is to review the query, the provisional answer and the reformulated query. Your objective is to judge the reformulated query and determine if it retains only those components of the original query that have not been addressed in the provisional answer.

The main objective of this reformulation is to avoid constraints that have already been addressed in previous attempts at solving the query, so rewrite the original query accordingly.

1. If the original query has components that have been wholly addressed in the provisional answer and thus ignored in the reformulated query, return "True".
2. If the original query has critical components that are important for the essence of the query, and those components have not been addressed in the provisional answer, yet they have still been ignored in the reformulated query, return "False".
3. If any component of the reformulated query directly contradicts its analogous component in the original query, return "False".
4. If the provisional answer does not provide a solution for any component but despite that, the reformulated query is not equivalent in essence to the original query, return "False".
5. If the provisional answer does not provide a solution for any component, and the reformulated query is equivalent to the original query with no contradictions, return "True".
6. If there is no provisional answer, and the reformulated query is equivalent to the original query with no contradictions and APIs to account for, return "True".
7. If the provisional answer is faulty, and the reformulated query is equivalent to the original query, return "True".

Your answer should only either True or False, and if the answer is False, then provide a reason. And you MUST call the Finish function.

A.2. System prompt for combining provisional answers into single final answer

You will be provided a query and a list of provisional answers provided by different attempts at solving it, bulleted by numbers.

Your task is to go through the list of answers and combine them into one meaningful answer to the query provided.

You are not supposed to merely concatenate the answers, but instead logically combine elements so that they do not repeated.

Remember, your task is NOT to answer these queries based on your own knowledge. Your job is to simply combine the valid answers provided to you into a single coherent answer to the query such that it appears comprehensible to somebody who did not know about the provisional answers.

You must call the Finish function to finish.

Bibliography

- [1] OpenAI Platform. *GPT for Developers*. URL: <https://platform.openai.com/docs/overview>.
- [2] Aakanksha Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. 2022. arXiv: 2204.02311 [cs.CL]. URL: <https://arxiv.org/abs/2204.02311>.
- [3] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: <https://arxiv.org/abs/2302.13971>.
- [4] Lei Huang et al. *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. 2023. arXiv: 2311.05232 [cs.CL].
- [5] Thi Thumasathit. *Why LLM Fine-Tuning isn't Always a Good Idea*. URL: <https://gleen.ai/blog/llm-fine-tuning/>.
- [6] ML6. *Large Language Models: to Fine-tune or not to Fine-tune?* URL: <https://www.ml6.eu/blogpost/fine-tuning-large-language-models>.
- [7] Patrick Lewis et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks." In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.
- [8] Matthias Biehl. *Bridging the gaps in LLMs with your APIs*. URL: https://www.softwareag.com/en_corporate/blog/llm-tool-usage.html.
- [9] Oded Ovadia et al. *Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs*. 2024. arXiv: 2312.05934 [cs.AI]. URL: <https://arxiv.org/abs/2312.05934>.
- [10] Indikaai. *LLM With Tools: A Survey — The Future of Language Models with Enhanced Capabilities*. 2024. URL: <https://www.indikaai.com/blog/llm-with-tools-a-survey---the-future-of-language-models-with-enhanced-capabilities>.
- [11] Seungjae Ryan Lee. *TALM: Tool Augmented Language Models*. 2023. URL: <https://www.ryanlee.ai/blog/talm-tool-augmented-language-models/>.
- [12] Lilian Weng. *LLM Powered Autonomous Agents*. 2023. URL: <https://lilianweng.github.io/posts/2023-06-23-agent/>.
- [13] Yule Wang. *A Complete Guide to LLMs-based Autonomous Agents (Part I)*. 2023. URL: <https://medium.com/the-modern-scientist/a-complete-guide-to-llms-based-autonomous-agents-part-i-69515c016792>.

- [14] Eric Jang. “Can LLMs Critique and Iterate on Their Own Outputs?” In: *evjang.com* (Mar. 2023). URL: <https://evjang.com/2023/03/26/self-reflection.html>.
- [15] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI].
- [16] Shi Yu et al. *Few-Shot Generative Conversational Query Rewriting*. 2020. arXiv: 2006.05009 [cs.IR]. URL: <https://arxiv.org/abs/2006.05009>.
- [17] Svitlana Vakulenko et al. “Question Rewriting for Conversational Question Answering.” In: *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. WSDM ’21. Virtual Event, Israel: Association for Computing Machinery, 2021, pp. 355–363. ISBN: 9781450382977. DOI: 10.1145/3437963.3441748. URL: <https://doi.org/10.1145/3437963.3441748>.
- [18] Tejas Bawaskar. *Deep Learning: Autocorrect, Spell check for short words*. 2020. URL: <https://towardsdatascience.com/deep-learning-autocorrect-product-and-technical-overview-1c219cee0698>.
- [19] Arunprasath Shankar and Venkata Nagaraju Buddarapu. “Legal Query Reformulation using Deep Learning.” In: *ASAIL@ ICAIL* (2019). URL: <https://ceur-ws.org/Vol-2385/paper4.pdf>.
- [20] Xiao Wang et al. “Generative query reformulation for effective adhoc search.” In: *arXiv preprint arXiv:2308.00415* (2023).
- [21] Jie Liu and Barzan Mozafari. *Query Rewriting via Large Language Models*. 2024. arXiv: 2403.09060 [cs.DB]. URL: <https://arxiv.org/abs/2403.09060>.
- [22] Yunfan Gao et al. *Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks*. 2024. arXiv: 2407.21059 [cs.CL]. URL: <https://arxiv.org/abs/2407.21059>.
- [23] Xiaonan Li et al. “LLattribution: LLM-Verified Retrieval for Verifiable Generation.” In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 5453–5471. URL: <https://aclanthology.org/2024.naacl-long.305>.
- [24] Luyu Gao et al. “RARR: Researching and Revising What Language Models Say, Using Language Models.” In: *ArXiv* abs/2210.08726 (2022). URL: <https://arxiv.org/abs/2210.08726>.
- [25] Weijia Shi et al. *REPLUG: Retrieval-Augmented Black-Box Language Models*. 2023. arXiv: 2301.12652 [cs.CL]. URL: <https://arxiv.org/abs/2301.12652>.

- [26] Ori Ram et al. “In-Context Retrieval-Augmented Language Models.” In: *Transactions of the Association for Computational Linguistics* 11 (2023), pp. 1316–1331. DOI: 10.1162/tac1_a_00605. URL: <https://aclanthology.org/2023.tac1-1.75>.
- [27] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. arXiv: 2303.17651 [cs.CL].
- [28] Zhihong Shao et al. “Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy.” In: *arXiv preprint arXiv:2305.15294* (2023).
- [29] Liang Wang, Nan Yang, and Furu Wei. “Query2doc: Query expansion with large language models.” In: *arXiv preprint arXiv:2303.07678* (2023).
- [30] Fengran Mo et al. “ConvGQR: Generative Query Reformulation for Conversational Search.” In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 2023, pp. 4998–5012.
- [31] Kaustubh D. Dhole and Eugene Agichtein. “GenQREnsemble: Zero-Shot LLM Ensemble Prompting for Generative Query Reformulation.” In: *Advances in Information Retrieval*. Springer Nature Switzerland, 2024, pp. 326–335. ISBN: 9783031560637. DOI: 10.1007/978-3-031-56063-7_24. URL: http://dx.doi.org/10.1007/978-3-031-56063-7_24.
- [32] Adyasha Maharana et al. “Evaluating Very Long-Term Conversational Memory of LLM Agents.” In: *arxiv* (2024).
- [33] Bhargavi Paranjape et al. *ART: Automatic multi-step reasoning and tool-use for large language models*. 2023. arXiv: 2303.09014 [cs.CL]. URL: <https://arxiv.org/abs/2303.09014>.
- [34] Timo Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 2023. arXiv: 2302.04761 [cs.CL]. URL: <https://arxiv.org/abs/2302.04761>.
- [35] Silin Gao et al. *Efficient Tool Use with Chain-of-Abstraction Reasoning*. 2024. arXiv: 2401.17464 [cs.CL]. URL: <https://arxiv.org/abs/2401.17464>.
- [36] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. 2023. arXiv: 2307.16789 [cs.AI].
- [37] Shishir G. Patil et al. “Gorilla: Large Language Model Connected with Massive APIs.” In: *arXiv preprint arXiv:2305.15334* (2023).
- [38] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [39] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” In: *arXiv preprint arXiv:2210.03629* (2022).

- [40] Mike D’Arcy et al. “Marg: Multi-agent review generation for scientific papers.” In: *arXiv preprint arXiv:2401.04259* (2024).
- [41] OpenAI Platform. *Function calling*. URL: <https://platform.openai.com/docs/guides/function-calling>.
- [42] Yu Du, Fangyun Wei, and Hongyang Zhang. “AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls.” In: *arXiv preprint arXiv:2402.04253* (2024).
- [43] LangChain. *Agents*. URL: <https://python.langchain.com/v0.1/docs/modules/agents/>.
- [44] Tony Kipkemboi and Sarah Chieng. *Build a Multi-AI Agent Workflow with Cerebras and CrewAI*. URL: <https://platform.openai.com/docs/guides/function-calling>.
- [45] Nelson F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172. 2023.
- [46] OpenAI Platform. *Models*. URL: <https://platform.openai.com/docs/models>.
- [47] Ted Hessing. *Two Sample Z Test of Proportions*. URL: <https://sixsigmastudyguide.com/two-sample-test-of-proportions/>.